Matrix Completion and Recommender Systems

Guillaume Lecué*

Abstract

In this chapter we introduce the matrix completion problem, which has been studied in particular for the design of recommender systems. A natural sparsity assumption in this problem is the *low-rank* assumption. The procedure naturally associated with this assumption is based on minimizing the rank function over a solution space $\{B \in \mathbb{R}^{u\times v} : \langle B, X_i\rangle = y_i, \forall i = 1,\dots,m\}$. This procedure is NP-hard in general. We then use the *convex relaxation* approach in this setting. The resulting procedure consists in minimizing the nuclear norm (i.e., the sum of the singular values) over the solution space. We show that this procedure can be rewritten as a *semidefinite programming* problem and can therefore be implemented (more or less efficiently, depending on the size of the matrix to be completed).

We then analyze the nuclear norm minimization procedure by the *RIP approach*, and then by a more general approach based on the study of the size of the subdifferential of the regularization norm and on a lower bound for a quadratic process.

1 Introduction

Let $A^* \in \mathbb{R}^{u\times v}$. We assume that we are given m linear measurements of A^* of the form $y_i = \langle A^*, X_i\rangle$, $i = 1,\dots,m$. We wish to reconstruct A^* exactly from the measurements $(y_i)_{i=1}^m$ and from the projection directions $X_i \in \mathbb{R}^{u\times v}$.

Since there are uv unknowns (the number of entries of A^*) and only $m << uv$ equations, we recover a highly underdetermined problem, as in the previous chapters. We nevertheless know that this type of problem can be solved if the signal A^* to be reconstructed is structured in such a way that its true dimension is in fact not uv, but a much smaller quantity (in particular, smaller than m). The structure that arises naturally in many problems of this type (as we shall see below for the design of recommender systems) is a low-rank structure: we assume that A^* has rank $r << u \wedge v$. The low-rank assumption may then be viewed as our sparsity assumption when we wish to reconstruct a matrix from a small number of linear measurements.

Key point : For problems of reconstructing a matrix from a small number of linear measurements of that matrix, the low-rank assumption is an assumption that makes it possible to solve the problem.

Recommender systems by completion of the users/items matrix. Consider, for example, the problem associated with recommender systems. There are several ways of viewing (and hence of solving) this problem. The viewpoint adopted here (and which has been very successful, in particular following the Netflix Prize) is that of matrix completion; the matrix we seek to complete is the **users/items** matrix. Completing the users/items matrix yields a list of items to recommend first to each user.

*ESSEC, Business school. Email: lecue@essec.edu.

For the Netflix Prize, one had $u = 480.189$ users and $v = 17.770$ items (here, movies). Users rate certain movies with a score ranging from 1 to 5 (for example, 1 when the user did not like the movie and 5 when they liked it very much). Not every user has rated every movie; in fact, out of the maximum number uv of possible ratings, only 100.480.507 ratings were obtained, that is, $1,2\%$ of the total number of ratings.

A useful way (at least from the point of view of matrix completion) of representing these data is to consider the $u \times v$ matrix called the users/items matrix, whose (i, j) entry is the rating of user i for item j when it exists (that is, when user i has rated item j), and which leaves that entry empty when the rating is unknown (see Figure 1).

Item/ User	10	11	13	14	15	16	17	18	19
1	2	3	4	5					
2		6			6	1	10		
3		4	1		7			2	
4		8	1						5
5		2							
6							5	1	

⇓ Matrix Completion ⇓

Item/ User	10	11	13	14	15	16	17	18	19
1	2	3	4	5	2	4	2	3	1
2	4	6	3	4	6	1	10	9	3
3	7	4	1	3	7	7	2	2	9
4	6	8	1	4	6	10	2	3	5
5	1	2	7	2	9	2	9	9	2
6	7	8	2	10	2	3	5	1	3

Figure 1: The users/items matrix completed by a matrix completion algorithm for the design of a recommender system.

Once the users/items matrix has been constructed for a given problem, one can run a matrix completion algorithm on this matrix in order to fill in the empty entries. Once the matrix has been completed, one can identify, on each row (that is, for each user), the item or items with the highest ratings, and thus recommend that item or those items to that user. This is how completion algorithms can be used to build recommender systems. There are many matrix completion algorithms; we shall see a few of them in this chapter.

Key point : One can build a recommender system by completing the users/items matrix.

Low-rank assumption. Constructing the users/items matrix is only a way of formatting the data; it makes it possible to visualize concisely what one is trying to do, and it can suggest how to do it. A formal way of writing these data is to view them as observations of linear projections of a matrix A^*, as at the beginning of the chapter. Indeed, if user p has rated item q with the rating $y_{p,q}$, then one may view $y_{p,q}$ as the value of the inner product $\langle A^*, E_{p,q} \rangle$, where $E_{p,q}$ is the matrix with all entries equal to 0 except at (p, q), where it equals 1. We therefore have $A^*_{p,q} = \langle A^*, E_{p,q} \rangle = y_{p,q}$. Thus, for

$$\Omega = \{(p, q) \in \{1, \dots, u\} \times \{1, \dots, v\} \text{ such that user } p \text{ has rated item } q\}$$

we observe all the entries of A^* indexed by Ω, and the matrix to be completed in Figure 1 is therefore represented by $(A^*_{p,q} : (p,q) \in \Omega)$. The problem of completing the users/items matrix is thus indeed equivalent to the problem of reconstructing a matrix A^* from a small number of linear measurements of that matrix.

This problem does not have a unique solution (there is an affine space of matrices A such that $\langle A, X_i \rangle = y_i$, $i = 1, \ldots, m$). However, the matrix A^* that we wish to reconstruct is structured: we believe that it has low rank (or is approximately low rank, that is, close to a low-rank matrix). Indeed, in the case of the problem posed by Netflix (and of recommender systems in general), one imagines that users can be classified according to a relatively small number of profiles, such as fans of science fiction, of crime films, of adventure films, and so on; likewise, one assumes that movies can be well classified according to themes. Thus the nearly 500.000 Netflix users form (more or less) homogeneous groups, which therefore tend to assign (more or less) the same ratings to movies in the same categories. Intuitively, the matrix A^* that we seek to reconstruct should be close to a block matrix in which all science-fiction fans have given a 5 to the good science-fiction movies, adventure fans have given a 5 to the good adventure movies, and so on. This block matrix has low rank; its rank is of the order of the minimum between the number of homogeneous groups of users and the number of homogeneous groups of items.

It is in this way that the low-rank assumption entered the recommender-system problem: if one believes that the users and/or the items have this structure of homogeneous groups, then one may expect the matrix A^* to be close to a low-rank matrix, and the assumption therefore seems justified. In what follows, we shall make the approximation that A^* has low rank (and is not merely approximately low rank), just as in the previous chapters we assumed that the vectors to be reconstructed were exactly sparse. The more general study of approximately low-rank matrices, as well as of noisy data, requires no new ideas, which is why we consider only this simpler setting.

Key point : The users/items matrix is approximately low rank. This property is useful for the design of algorithms that complete this matrix (and hence of recommender systems).

2 Convex relaxation in matrix completion

The problem introduced above, which we wish to solve, is the following: given m linear measurements $y_i = \langle A^*, X_i \rangle$, $i = 1, \ldots, m$ of a low-rank matrix $A^* \in \mathbb{R}^{u \times v}$ in the directions $X_1, \ldots, X_m \in \mathbb{R}^{u \times v}$, how can one reconstruct A^* exactly? How many measurements m are needed? How should one choose the measurement matrices $X_1, \ldots, X_m$? And, in the particular case of the matrix completion problem, how should one choose the X_i in the set $\{E_{p,q} : 1 \leq p \leq u, 1 \leq q \leq v\}$?

The aim of this section is to follow the approach used in the previous chapters on the reconstruction of a sparse vector, for the problem we have here, namely the problem of reconstructing a low-rank matrix from a small number of linear measurements.

We begin with a few **notations** that will be useful below. On the space $\mathbb{R}^{u \times v}$ of matrices of size $u \times v$, we define an inner product by

$$\langle A, B \rangle = \sum_{i,j} A_{ij} B_{ij}.$$

For every matrix $A \in \mathbb{R}^{u \times v}$, we denote by σ_A its **spectrum**, that is, the vector in $\mathbb{R}^{u \wedge v}$ of singular values (which are the square roots of the eigenvalues of $A^\top A$): $\sigma_A = (\sigma_j)_{j=1}^{u \wedge v}$ and $\sigma_1(A) \geq \ldots, \geq$

$\sigma_{u\wedge v}(A) \geq 0$. We then define the **Schatten norms** by

$$\|A\|_{S_p} = \|\sigma_A\|_{\ell_p} = \Big(\sum_{j=1}^{u\wedge v} \sigma_j(A)^p\Big)^{1/p}$$

for every $A \in \mathbb{R}^{u\times v}$ and $p \geq 1$. The rank of a matrix A is the dimension of its image; it is also the number of nonzero singular values, and can therefore be expressed as the ℓ_0 "norm" of its spectrum:

$$\operatorname{rang}(A) = \|\sigma_A\|_{\ell_0}\,.$$

The S_1 norm is also called the **nuclear norm**, or **trace norm**. The S_2 norm is the **Frobenius norm**; it is the Hilbert norm associated with the inner product $\langle\cdot,\cdot\rangle$ defined above. The S_∞ norm is the **operator norm** from ℓ_2^v to ℓ_2^u:

$$\|A\|_{S_\infty} = \sup_{\|x\|_2=1} \|Ax\|_2\,.$$

We recall the duality relations

$$\|A\|_{S_q} = \sup\big(\langle A, B\rangle : \|B\|_{S_p} \leq 1\big)$$

when $q^{-1} + p^{-1} = 1$.

For every integer p, we write

$$\mathcal{S}^p = \{A \in \mathbb{R}^{p\times p} : A^\top = A\}, \mathcal{S}^p_+ = \{A \in \mathcal{S}^p : A \succeq 0\} \text{ and } \mathcal{S}^p_{++} = \{A \in \mathcal{S}^p : A \succ 0\}$$

which are, respectively, the space of **symmetric matrices**, the cone of **positive semidefinite matrices** (that is, such that $x^\top Ax \geq 0$ for every x), and the cone of **positive definite matrices** (that is, such that $x^\top Ax > 0$ for every $x \neq 0$). We also denote the group of **orthogonal matrices** by

$$\mathcal{O}(p) = \{A \in \mathbb{R}^{p\times p} : A^\top A = AA^\top = I_p\}$$

where I_p is the identity matrix of $\mathbb{R}^p$.

2.1 Rank minimization procedure.

Under the low-rank assumption, a natural procedure for reconstructing A^* from the measurements $y_i = \langle X_i, A^*\rangle$, $i = 1,\dots,m$, consists in seeking a solution of minimal rank in the space $\{A \in \mathbb{R}^{u\times v} : \langle A, X_i\rangle = y_i, i = 1,\dots,m\}$ of all solutions with the same projections:

$$\min\Big(\operatorname{rang}(A) : \langle A, X_i\rangle = y_i, i = 1,\dots,m\Big). \tag{2.1}$$

One could study the theoretical properties of the procedure (2.1) in the same way as we studied the ℓ_0 minimization procedure in the previous chapters. However, the ℓ_0 minimization procedure was ruled out by its algorithmic properties. Indeed, we showed that ℓ_0 minimization is NP-hard in general. This last point justified setting that procedure aside, and it is also this last point that leads us to abandon the procedure (2.1) at once.

Proposition 2.1. *Finding a solution to the problem of rank minimization over an affine space is an NP-hard problem in general.*

Proof. We use a reduction principle, as we did when reducing the ℓ_0 minimization problem to the problem of covering by sets of size 3. Here, we show that the rank minimization problem can be reduced to the problem of minimizing the ℓ_0 norm in polynomial time. The idea is that if one has an algorithm solving problems of the form

$$\min_{\mathcal{A}(A)=y} \operatorname{rang}(A) \tag{2.2}$$

where $y \in \mathbb{R}^m$ and $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$ is a linear operator, then one will also have an algorithm solving problems of the form

$$\min_{Bt=b} \|t\|_0 \tag{2.3}$$

where $B \in \mathbb{R}^{m_0\times N}$ and $b \in \mathbb{R}^{m_0}$.

Indeed, consider a problem of the form (2.3) for a matrix $B \in \mathbb{R}^{m_0\times N}$ and a vector $b \in \mathbb{R}^{m_0}$. Set $u = v = N$ and $m = m_0 + N^2$. We construct the operator

$$\mathcal{A} : \begin{cases} \mathbb{R}^{u\times v} & \longrightarrow & \mathbb{R}^{m_0+u\times v} \\ A & \longrightarrow & \begin{pmatrix} B\operatorname{diag}(A) \\ A - \operatorname{diag}(A) \end{pmatrix} \end{cases}$$

where u and v are such that $u \wedge v = N$ and $\operatorname{diag}(A) = (a_{ii})_{1\leq i\leq N} \in \mathbb{R}^N$. We construct the measurement vector

$$y = \begin{pmatrix} b \\ 0 \end{pmatrix} \in \mathbb{R}^{m_0+N^2}.$$

For this operator $\mathcal{A}$ and this measurement vector y, one sees that the solution space satisfies

$$\big\{A \in \mathbb{R}^{u\times v} : \mathcal{A}(A) = y\big\} = \big\{\text{diagonal matrices of } \mathbb{R}^{u\times v} \text{ with diagonal } t : Bt = b\big\}$$

and, since the rank of a diagonal matrix is the ℓ_0 "norm" of its diagonal, if $\hat{A}$ is a solution of (2.2) for the choice of $\mathcal{A}$ and y as above, then $\hat{A}$ is a diagonal matrix and its diagonal $\hat{t} = \operatorname{diag}(\hat{A})$ is a solution of (2.3). Thus, since the construction of $\mathcal{A}$ and y from B and b is carried out in time polynomial in the input dimensions of B and b, one sees that the existence of an algorithm solving (2.3) can be reduced in polynomial time to an algorithm solving (2.3). Since (2.3) is NP-hard, so is (2.2). ∎

The rank minimization procedure is therefore NP-hard in general. This means that, unless one falls into a rather favorable situation, one cannot find an algorithm that solves this problem exactly in a reasonable time. We shall therefore not use it in practice, and hence not use it as is.

Nevertheless, one can try to understand what makes the problem (2.2) so difficult numerically. We see here that the constraint is an affine space; it is therefore a constraint that is fairly easy to handle (for example, one knows how to project onto it quite easily). It is therefore the objective function $A \to \operatorname{rang}(A)$ that is problematic. In particular, it is not convex. A natural approach, similar to the one used to construct Basis Pursuit, is to replace the (nonconvex) objective function $A \to \operatorname{rang}(A)$ by the "closest convex function". This is the approach known as convex relaxation, which we shall now carry out for the problem (2.2).

2.2 Convex envelope of the rank

In this section, we replace the objective function $A \to \operatorname{rang}(A)$ of the optimization problem (2.2), namely rank minimization over the solution space of the linear system, by a convex function that is close to it. We first recall a way of convexifying a function on a set.

Definition 2.2. *Let E be a vector space and C a convex subset of E. Let $f : C \to \mathbb{R}$. The* ***convex envelope of f on C*** *is the largest convex function that is smaller than f on C. It is denoted by* $\mathrm{conv}(f)$.

In other words, for every $x \in C$,

$$\mathrm{conv}(f) : x \in C \to \sup\big(g(x) : g \text{ is convex on } C \text{ and } g(y) \leq f(y), \forall y \in C\big).$$

This definition is meaningful because the pointwise maximum of a family of convex functions is convex. This is therefore the case for the maximum of the set of all convex functions on C that are smaller than f uniformly on C.

When E is a Hilbert space, one can show, by geometric arguments on the epigraph of f, or by writing that the convex function $\mathrm{conv}(f)$ is the maximum of its tangents, that one may restrict the definition of $\mathrm{conv}(f)$ to all affine functions smaller than f: for every $x \in C$,

$$\mathrm{conv}(f)(x) = \sup_{a \in H, b \in \mathbb{R}} \big(\langle a, x\rangle + b : \langle a, y\rangle + b \leq f(y), \forall y \in C\big).$$

Definition 2.3. *Let H be a Hilbert space and C a convex subset of H. Let $f : C \to \mathbb{R}$. The* ***Fenchel transform of f on C*** *is the function defined at every point $x \in H$ by*

$$f^*(x) = \sup_{y \in C} \big(\langle x, y\rangle - f(y)\big).$$

The Fenchel transform of f is a convex function (even if f is not convex), because it is the maximum of a family of linear (hence convex) functions. It is defined on the whole space H. One may then define the Fenchel transform of f^* on H by: for every $y \in H$,

$$f^{**}(y) = \sup_{x \in H} \big(\langle x, y\rangle - f^*(x)\big).$$

The function f^{**} is called the **biconjugate** of f. Note that the supremum defining f^{**} is taken over all of H, whereas the supremum defining f^* is restricted to C.

Theorem 2.4. *Let H be a Hilbert space and C a convex subset of H. Let $f : C \to \mathbb{R}$. Denote by* $\mathrm{conv}(f)$ *the convex envelope of f on C, and by f^{**} the biconjugate of f on C. Then, for every $x \in C$, $f^{**}(x) = \mathrm{conv}(f)(x)$.*

Proof. We use the characterization of $\mathrm{conv}(f)$ in terms of affine functions: for every $x \in C$,

$$\mathrm{conv}(f)(x) = \sup_{a \in H, b \in \mathbb{R}} \big(\langle a, x\rangle + b : \langle a, y\rangle + b \leq f(y), \forall y \in C\big). \tag{2.4}$$

Given $a \in H$, one sees that the constraint in (2.4) is satisfied for every $b \in \mathbb{R}$ such that, for every $y \in C$,

$$b \leq f(y) - \langle a, y\rangle.$$

Since we seek to maximize in a and b, we have no choice but to take

$$b = \inf_{y \in C} \big(f(y) - \langle a, y\rangle\big) = -f^*(a).$$

For this choice of b the constraint is satisfied, and we therefore have

$$\mathrm{conv}(f)(x) = \sup_{a \in H} \big(\langle a, x\rangle - f^*(a)\big) = f^{**}(x).$$

■

It therefore suffices to compute the **biconjugate** of f in order to know its convex envelope. This is the strategy we use to determine the convex envelope of the rank on the unit ball of the operator norm.

Theorem 2.5. *The convex envelope of the rank function on $B_{S_\infty} = \{A \in \mathbb{R}^{u\times v} : \|A\|_{S_\infty} \leq 1\}$ is the nuclear norm.*

Proof. We begin by recalling von Neumann's inequality.

Proposition 2.6 (von Neumann's inequality)**.** *Let $A, B \in \mathbb{R}^{u\times v}$. Then*

$$|\langle A, B\rangle| \leq \sum_i \sigma_i(A)\sigma_i(B)$$

where $\sigma_1(A) \geq \cdots \geq \sigma_{u\wedge v}(A)$ is the nonincreasing sequence of singular values of A. Equality $\langle A, B\rangle = \sum_i \sigma_i(A)\sigma_i(B)$ holds if and only if, given the SVD $A = U_A D_A V_A^\top$ of A, one has $U_A^\top B V_A = D_B$, where $D_A = \operatorname{diag}(\sigma_A)$ and $D_B = \operatorname{diag}(\sigma_B)$.

Likewise, $\|A - B\|_{S_2} \geq \|\sigma_A - \sigma_B\|_2$, and equality holds when $B = U_A D_B V_A^\top$.

Part 1: We compute the Fenchel transform of the rank function $f(A) = \operatorname{rang}(A)$ on B_{S_∞}. For every $A \in \mathbb{R}^{u\times v}$,

$$f^*(A) = \sup_{B\in B_{S_\infty}} \big(\langle A, B\rangle - f(B)\big).$$

By von Neumann's inequality, $\langle A, B\rangle \leq \sum \sigma_i(A)\sigma_i(B)$, and since the rank and the S_∞ norm are invariant under left and right multiplication by orthogonal matrices, we have

$$f^*(A) = \sup_{B\in B_{S_\infty}} \big(\sum_i \sigma_i(A)\sigma_i(B) - f(B)\big).$$

We write $q = u \wedge v$ and then partition B_{S_∞} according to the rank:

$$\begin{aligned} f^*(A) &= \sup_{B\in B_{S_\infty}} \big(\sum_i \sigma_i(A)\sigma_i(B) - f(B)\big) = \max_{0\leq r\leq q} \sup_{B\in B_{S_\infty}: f(B)=r} \big(\sum_{i=1}^r \sigma_i(A)\sigma_i(B) - r\big) \\ &= \max_{0\leq r\leq q} \sup_{B\in B_{S_\infty}: f(B)=r} \big(\sum_{i=1}^r \sigma_i(A) - r\big) = \sum_{i=1}^q (\sigma_i(A) - 1)_+ \end{aligned}$$

Part 2: We now compute the biconjugate of f. For every $B \in \mathbb{R}^{u\times v}$,

$$f^{**}(B) = \sup_{A\in\mathbb{R}^{u\times v}} \Big(\langle A, B\rangle - f^*(A)\Big).$$

It follows from von Neumann's inequality and from the equality case that

$$f^{**}(B) = \sup_A \Big(\sum \sigma_i(A)\sigma_i(B) - f^*(A)\Big).$$

For every $\|A\|_{S_\infty} \leq 1$, we have

$$f^*(A) = \sum_i (\sigma_i(A) - 1)_+ = 0.$$

We therefore have

$$\sup_{\|A\|_{S_\infty}\leq 1} \Big(\sum \sigma_i(A)\sigma_i(B) - f^*(A)\Big) = \sup_{\|A\|_{S_\infty}\leq 1} \sum \sigma_i(A)\sigma_i(B) = \sum \sigma_i(B) = \|B\|_{S_1}.$$

Moreover, if $\|A\|_{S_\infty} \geq 1$ and if $\|B\|_{S_\infty} \leq 1$, we have

$$\begin{aligned}
&\sum \sigma_i(A)\sigma_i(B) - f^*(A) = \sum_i \sigma_i(A)\sigma_i(B) - \sum_i (\sigma_i(A)-1)_+ \\
&= \sum_i \sigma_i(B) + \sum_i (\sigma_i(A)-1)\sigma_i(B) - \sum_i (\sigma_i(A)-1)_+ \\
&= \|B\|_{S_1} + \underbrace{\sum_{i:\sigma_i(A)\leq 1} (\sigma_i(A)-1)\sigma_i(B)}_{\leq 0} + \underbrace{\sum_{i:\sigma_i(A)>1} (\sigma_i(A)-1)(\sigma_i(B)-1)}_{\leq 0} \leq \|B\|_{S_1}
\end{aligned}$$

and we conclude because $\sigma_i(B) \leq 1$ for every i. ∎

Conclusion : The convex envelope of the rank function on B_{S_∞} is the nuclear norm. It is then natural to "replace" the rank objective in the NP-hard rank minimization problem by the nuclear norm, according to the principle of convex relaxation.

2.3 Nuclear norm minimization procedure and SDP problem

The convex relaxation of problem (2.1) is therefore the procedure that minimizes the nuclear norm over the solution space:

$$\hat{A} \in \operatorname*{argmin}_{A\in\mathbb{R}^{u\times v}} \big(\|A\|_{S_1} : \mathcal{A}(A) = y \big). \tag{NM}$$

where we write $\mathcal{A}(A) = (\langle A, X_i \rangle)_{i=1}^m$ and $y = (y_i)_{i=1}^m$.

As for Basis Pursuit, one checks that nuclear norm minimization over the solution space can be implemented efficiently by rewriting (NM) as a *semidefinite programming* **(SDP)** problem (we recall the definition of an SDP problem in standard form in Definition 2.9).

To this end, we consider the following problem

$$\begin{pmatrix} \hat{X} \\ \hat{Y} \\ \hat{Z} \end{pmatrix} \in \operatorname*{argmin}_{X\in\mathbb{R}^{u\times v}, Y\in\mathcal{S}^u, Z\in\mathcal{S}^v} \operatorname{Tr}(Y) + \operatorname{Tr}(Z) \text{ such that } \begin{array}{c} \mathcal{A}(X) = y \\ \begin{bmatrix} Y & X \\ X^\top & Z \end{bmatrix} \succeq 0 \end{array} \tag{SDP}$$

Theorem 2.7. *The two problems (NM) and (SDP) are equivalent:*

1. *if $\hat{A}$ is a solution of (NM), then $(\hat{A}, \hat{Y}, \hat{Z})^\top$ is a solution of (SDP) for some $\hat{Y}$ and $\hat{Z}$ (depending only on $\hat{A}$; cf. (2.5)),*
2. *if $(\hat{X}, \hat{Y}, \hat{Z})^\top$ is a solution of (SDP), then $\hat{X}$ is a solution of (NM).*

To prove Theorem 2.7, we first prove the following lemma.

Lemma 2.8. *Let $A \in \mathbb{R}^{u\times v}$ and $t \geq 0$. The following two assertions are equivalent:*

a) $\|A\|_{S_1} \leq t$

b) there exist $Y \in \mathcal{S}^u$, $Z \in \mathcal{S}^v$ such that

$$\operatorname{Tr}(Y) + \operatorname{Tr}(Z) \leq 2t \text{ and } \begin{bmatrix} Y & A \\ A^\top & Z \end{bmatrix} \succeq 0$$

Proof. Assume that *b)* holds. Consider the SVD of A: $A = UDV^\top$, where $U \in \mathcal{O}(u)$, $D = \mathrm{diag}(\sigma_A) \in \mathbb{R}^{u\times v}$, and $V \in \mathcal{O}(v)$. One sees that for $I_{u,v} = \mathrm{diag}((1)_{i=1}^q) \in \mathbb{R}^{u\times v}$, where $q = u \wedge v$, the matrix

$$\begin{pmatrix} I_u & -UI_{u,v}V^\top \\ -VI_{u,v}^\top U^\top & I_v \end{pmatrix}$$

is positive semidefinite, because for every $x \in \mathbb{R}^u$, $y \in \mathbb{R}^v$, we have

$$\begin{pmatrix} x^\top & y^\top \end{pmatrix} \begin{pmatrix} I_u & -UI_{u,v}V^\top \\ -VI_{u,v}^\top U^\top & I_v \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \|x\|_2^2 + \|y\|_2^2 - 2\langle U^\top x, I_{u,v}V^\top y\rangle \geq 0.$$

Since the product of two positive semidefinite matrices is also positive semidefinite, we have

$$\mathbf{Tr}\left[\begin{pmatrix} I_u & -UI_{u,v}V^\top \\ -VI_{u,v}^\top U^\top & I_v \end{pmatrix} \begin{pmatrix} Y & A \\ A^\top & Z \end{pmatrix}\right] \geq 0$$

that is,

$$\mathbf{Tr}(Y - UI_{u,v}V^\top A^\top) + \mathbf{Tr}(-VI_{u,v}U^\top A + Z) \geq 0$$

and hence $2t \geq \mathbf{Tr}(Y) + \mathbf{Tr}(Z) \geq 2\mathbf{Tr}(D) = 2\|A\|_{S_1}$.

Assume that *a)* holds. Consider the SVD of A in the form $A = U_r D_r V_r^\top$, where $r = \mathrm{rang}(A)$, $U_r \in \mathbb{R}^{u\times r}$ is the matrix whose columns are the left singular vectors of A, $V_r \in \mathbb{R}^{v\times r}$ is the matrix whose columns are the right singular vectors of A, and $D_r \in \mathbb{R}^{r\times r}$ is the diagonal matrix of the nonzero singular values of A.

We set

$$Y = U_r D_r U_r^\top + \gamma I_u \in \mathcal{S}^u \text{ and } Z = V_r D_r V_r^\top + \gamma I_v \in \mathcal{S}^v \tag{2.5}$$

for some $\gamma > 0$ to be determined. We have

$$\mathbf{Tr}(Y) + \mathbf{Tr}(Z) = 2\mathbf{Tr}(D_r) + \gamma(u+v) = 2\|A\|_{S_1} + \gamma(u+v) = 2t$$

for $\gamma = 2(t - \|A\|_{S_1})/(u+v)$. Moreover,

$$\begin{bmatrix} Y & A \\ A^\top & Z \end{bmatrix} = \begin{bmatrix} U_r D_r U_r^\top & U_r D_r V_r^\top \\ V_r D_r U_r^\top & V_r D_r V_r^\top \end{bmatrix} + \gamma \begin{bmatrix} I_u & 0 \\ 0 & I_v \end{bmatrix} = \begin{bmatrix} U_r \\ V_r \end{bmatrix} D_r \begin{bmatrix} U_r^\top & V_r^\top \end{bmatrix} + \gamma I_{u+v} \succeq 0$$

■

Proof of Theorem 2.7. We first prove the first point. If $\hat{A}$ is a solution of (NM), then by Lemma 2.8 there exist $\hat{Y}$, $\hat{Z}$ such that

$$\mathrm{Tr}(\hat{Y}) + \mathrm{Tr}(\hat{Z}) = 2\left\|\hat{A}\right\|_{S_1} \text{ and } \begin{bmatrix} \hat{Y} & \hat{A} \\ \hat{A}^\top & \hat{Z} \end{bmatrix} \succeq 0.$$

Since $\mathcal{A}(\hat{A}) = y$, we indeed have that $(\hat{A}, \hat{Y}, \hat{Z})$ lies in the constraint set of (SDP). Moreover, $\mathrm{Tr}(\hat{Y}) + \mathrm{Tr}(\hat{Z}) = 2\left\|\hat{A}\right\|_{S_1}$, so if (X, Y, Z) lies in the constraint set of (SDP), we shall have $\mathcal{A}(X) = y$ and hence, by optimality of $\hat{A}$, $\left\|\hat{A}\right\|_{S_1} \leq \|X\|_{S_1}$, but also $\mathrm{Tr}(Y) + \mathrm{Tr}(Z) = 2\|X\|_{S_1}$, and therefore necessarily

$$\mathrm{Tr}(\hat{Y}) + \mathrm{Tr}(\hat{Z}) = 2\left\|\hat{A}\right\|_{S_1} \leq 2\|X\|_{S_1} = \mathrm{Tr}(Y) + \mathrm{Tr}(Z).$$

Thus $(\hat{A}, \hat{Y}, \hat{Z})$ is indeed a solution of (SDP).

To prove point *2)*, assume that $(\hat{X}, \hat{Y}, \hat{Z})$ is a solution of (SDP). We have

$$\mathrm{Tr}(\hat{Y}) + \mathrm{Tr}(\hat{Z}) \leq 2t \text{ and } \begin{bmatrix} \hat{Y} & \hat{X} \\ \hat{X}^\top & \hat{Z} \end{bmatrix} \succeq 0$$

for $t = (\mathrm{Tr}(\hat{Y}) + \mathrm{Tr}(\hat{Z}))/2$. Then, by Lemma 2.8, we have

$$\left\|\hat{X}\right\|_{S_1} \leq \frac{\mathrm{Tr}(\hat{Y}) + \mathrm{Tr}(\hat{Z})}{2}.$$

Let A lie in the constraint space of (NM). By Lemma 2.8, there exist Y, Z such that $\mathrm{Tr}(Y) + \mathrm{Tr}(Z) = 2\|A\|_{S_1}$ and (A, Y, Z) lies in the constraint space of (SDP). In particular,

$$\left\|\hat{X}\right\|_{S_1} \leq \frac{\mathrm{Tr}(\hat{Y}) + \mathrm{Tr}(\hat{Z})}{2} \leq \frac{\mathrm{Tr}(Y) + \mathrm{Tr}(Z)}{2} = \|A\|_{S_1}$$

and, since $\mathcal{A}(\hat{X}) = y$, $\hat{X}$ lies in the constraint set of (NM). We therefore deduce that $\hat{X}$ is a solution of (NM). ∎

One can therefore rewrite the nuclear norm minimization problem as an *SDP* problem, whose general form we now recall.

Definition 2.9. *An optimization problem is a* ***semidefinite program (SDP)*** *if it can be rewritten in the form*

$$\min_{X \in \mathcal{S}^p} \left(\langle C, X \rangle : \tilde{\mathcal{A}}(X) = b, X \succeq 0 \right) \tag{2.6}$$

where $C \in \mathcal{S}^p$, $\tilde{\mathcal{A}} : \mathcal{S}^p \to \mathbb{R}^m$ is a linear map, and $b \in \mathbb{R}^m$.

One can rewrite problem (SDP) in the form of a *semidefinite program*:

$$\min \left(\langle C, \begin{bmatrix} Y & X \\ X^\top & Z \end{bmatrix} \rangle : \tilde{\mathcal{A}} \left(\begin{bmatrix} Y & X \\ X^\top & Z \end{bmatrix} \right) = y, \begin{bmatrix} Y & X \\ X^\top & Z \end{bmatrix} \succeq 0 \right)$$

where the minimum is taken over all symmetric matrices $\begin{bmatrix} Y & X \\ X^\top & Z \end{bmatrix}$,

$$C = I_{u+v}, \tilde{\mathcal{A}} \left(\begin{bmatrix} Y & X \\ X^\top & Z \end{bmatrix} \right) = \mathcal{A}(X) \text{ and } b = y.$$

There then exist algorithms similar to those seen for *linear programming*, which solve SDP problems approximately. For example, under `CVXOPT`, one solves (2.6) by turning C and X into vectors of size p^2 by concatenation (columnwise, for example) via the function $\mathrm{vec} : \mathbb{R}^{p \times p} \to \mathbb{R}^{p^2}$, and by rewriting $\tilde{\mathcal{A}}$ as a linear operator $A^\top : \mathbb{R}^{p^2}$ into $\mathbb{R}^m$. One then rewrites (2.6) in the form

$$\min_X \left(\langle \mathrm{vec}(C), \mathrm{vec}(X) \rangle : A^\top(\mathrm{vec}(X)) = b, X \succeq 0 \right)$$

which is solved by the command

```
> sol = cvxopt.solvers.conelp(-b, A, vec(C))
```

where $b \in \mathbb{R}^m$, $A \in \mathbb{R}^{m \times p^2}$, and $vec(C) \in \mathbb{R}^{p^2}$ are objects of type `matrix` in `CVXOPT`. One then recovers $\mathrm{vec}(X)$ by the command

```
> print(sol['z']).
```

Conclusion : One can rewrite the nuclear norm minimization problem as an SDP problem. There exist primal-dual barrier algorithms for solving SDPs.

3 Study of exact recovery by (NM)

The aim of this section is to determine for which measurement "vectors" $X_i \in \mathbb{R}^{u\times v}$, $i = 1, \dots, m$, and for which number m, one can reconstruct exactly every matrix A of rank r from the observations $y_i = \langle A, X_i \rangle$, $i = 1, \dots, m$, using the procedure (NM). Writing

$$\Sigma_r = \big\{ A \in \mathbb{R}^{u\times v} : \mathrm{rang}(A) = r \big\}$$

we are then interested in the following property:

Definition 3.1. *Let $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$ be a linear measurement operator $\mathcal{A}(A) = (\langle A, X_i \rangle)_{i=1}^m$. We say that $\mathcal{A}$ satisfies the **exact recovery property of order** r, denoted RE(r), when for every matrix $A \in \Sigma_r$,*

$$\operatorname*{argmin}_X \big(\|X\|_{S_1} : \mathcal{A}(X) = \mathcal{A}(A) \big) = \{A\}.$$

In other words, a measurement operator $\mathcal{A}(\cdot)$ satisfies RE(r) when, for every $A \in \Sigma_r$, there is a unique matrix of smallest nuclear norm in the solution space $\{X \in \mathbb{R}^{u\times v} : \mathcal{A}(X) = \mathcal{A}(A)\}$, and that matrix is A.

It is possible to repeat the same sequence of arguments as those developed for the study of Basis Pursuit, starting from matrix versions of the RIP and of the NSP. We briefly present these arguments (without proofs, because they are identical to the vector case) in the next section. In the last section we shall instead emphasize an alternative method of proof that highlights the role of the nondifferentiability of the objective function in inducing sparsity.

3.1 Argument based on the RIP approach

We present here, in the matrix case, the tools introduced earlier for the exact recovery of sparse vectors from a small number of measurements. We begin with the *Null Space Property*, which gives a necessary and sufficient condition for exact recovery.

Definition 3.2. *Let $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$ be a linear operator. We say that $\mathcal{A}$ satisfies the **rank Null Space Property (rank-NSP)** of degree r when, for every matrix A in the kernel of $\mathcal{A}$, one has*

$$\sum_{i=1}^{r} \sigma_i(A) < \sum_{i \geq r+1} \sigma_i(A)$$

where $\sigma_1(A) \geq \sigma_2(A) \geq \cdots$ is the nonincreasing sequence of singular values of A.

Proposition 3.3. *Let $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$ be a linear operator. The following two assertions are equivalent:*

1. *$\mathcal{A}$ satisfies RE(r)*
2. *$\mathcal{A}$ satisfies rank-NSP(r).*

It is nevertheless difficult to prove the rank-NSP directly. One may then consider a matrix version of the RIP.

Definition 3.4. *Let $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$ be a linear operator. We say that $\mathcal{A}$ satisfies the **rank Restricted Isometry Property (rank-RIP)** of degree r and with isometry constant δ_r when, for every matrix A of rank r, one has*

$$(1 - \delta_r) \|A\|_{S_2}^2 \leq \frac{\|\mathcal{A}(A)\|_{\ell_2^m}^2}{m} \leq (1 + \delta_r) \|A\|_{S_2}^2 .$$

Proposition 3.5. *There exist absolute constants $c_0 > 0$, $0 < c_1 < 1$, for which the following is true. Let $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$ be a linear operator. If $\mathcal{A}(\cdot)$ satisfies rank-RIP($c_0 r$) for $\delta_r < c_1$, then rank-NSP(r) holds.*

Moreover, one can show that in the case of Gaussian measurements, the rank-RIP(r) condition holds with high probability as soon as $m \gtrsim r(u+v)$. We first define what is meant by Gaussian measurements in the matrix case. We say that $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$, where $\mathcal{A}(A) = (\langle X_i, A\rangle)_{i=1}^m$, is a **Gaussian linear measurement operator** when, for every $i = 1, \ldots, m$, $X_i = (g_{pq} : 1 \le q \le u, 1 \le q \le v)$ is a Gaussian matrix, that is, the g_{pq} are i.i.d. $\mathcal{N}(0,1)$. Moreover, the X_i's are independent of one another.

Theorem 3.6. *Let $\mathcal{A} : \mathbb{R}^{u\times v} \to \mathbb{R}^m$ be a Gaussian measurement operator, and let $0 < \delta < 1$. There exist constants c_0, c_1, and c_2, depending only on δ, such that, with probability at least $1 - c_0 \exp(-c_1 m)$, $\mathcal{A}(\cdot)$ satisfies RIP(r) as soon as $m \ge c_2 r(u+v)$.*

One can also show, by a complexity argument, that $c_3 r(uv+v)$ measurements are necessary.

3.2 A general proof emphasizing the nondifferentiability of the objective function

In this section, we consider a Hilbert space equipped with its inner product $\langle \cdot, \cdot \rangle$. Let $E \subset H$ be a linear subspace of H, and let $\|\cdot\|$ be a norm on E. Let $X_1, \ldots, X_m$ be measurement vectors in H. We observe $y_i = \langle X_i, x^* \rangle$, $i = 1, \ldots, m$, for some $x^* \in E$. We wish to be able to reconstruct x^* exactly from the measurements $(y_i)_{i=1}^m$.

To this end, we consider the procedure

$$\hat{x} \in \operatorname{argmin} \big(\|x\| : \mathcal{A}(x) = y \big) \tag{3.1}$$

where $\mathcal{A}(x) = (\langle X_i, x\rangle)_{i=1}^m$ is the measurement operator and $y = (y_i)_{i=1}^m$ is the vector of measurements. We wish to know which x^* can be reconstructed exactly by (3.1), for which measurement vectors $X_1, \ldots, X_m$, and for which number m.

To answer these questions, we shall introduce a condition on the $X_1, \ldots, X_m$ and a condition on the subdifferential of the objective function defined by the norm $\|\cdot\|$. We introduce a few notations:

1. the dual norm associated with $\|\cdot\|$ is defined, for every $x \in E$, by

$$\|x\|_* = \sup \big(\langle x, y \rangle : \|y\| \le 1 \big),$$

2. the unit balls and spheres associated with $\|\cdot\|$, $\|\cdot\|_*$, and $\|\cdot\|_2$ (where $\|x\|_2^2 = \langle x, x\rangle$) are denoted, respectively, by

$$\begin{aligned} &B = \{x \in E : \|x\| \le 1\}, S = \{x \in E : \|x\| = 1\} \\ &B_* = \{x \in E : \|x\|_* \le 1\}, S_* = \{x \in E : \|x\|_* = 1\} \\ &B_2 = \{x \in H : \|x\|_2 \le 1\}, S_2 = \{x \in H : \|x\|_2 = 1\}. \end{aligned}$$

3. the subdifferential of $\|\cdot\|$ at x is

$$\partial \|\cdot\|(x) = \{g \in E : \|x+h\| \ge \|x\| + \langle g, h\rangle, \forall h \in E\} = \begin{cases} \{g \in S_* : \langle g, x\rangle = \|x\|\} & \text{if } x \ne 0 \\ B_* & \text{if } x = 0. \end{cases}$$

(cf. Example 2.5 of the previous chapter on proximal methods).

We first introduce a condition on the measurement operator.

Definition 3.7. *Let $X_1, \dots, X_m$ be vectors in H, and let $\mathcal{A} : H \to \mathbb{R}^m$ be the measurement operator such that $\mathcal{A}(A) = (\langle A, X_i \rangle)_{i=1}^m$. For every $r^* > 0$, we introduce the cone*

$$\mathcal{C}_{r^*} = \{x \in E : \|x\|_2 \geq r^* \|x\|\}. \tag{3.2}$$

*We say that $\mathcal{A}(\cdot)$ satisfies the **restricted singular value condition of radius** r^*, denoted VSR(r^*), when for every $x \in \mathcal{C}_{r^*}$ one has*

$$\frac{1}{m} \sum_{i=1}^m \langle x, X_i \rangle^2 \geq \frac{\|x\|_2^2}{2}. \tag{3.3}$$

When $H = \mathbb{R}^N$ and $\|\cdot\| = \|\cdot\|_1$, one shows that VSR(r^*) is implied by RIP(s) for $r^* \sim 1/\sqrt{s}$. For example, when the measurements are sub-Gaussian, one can show that with probability at least $1 - 2\exp(-c_0 m)$, VSR(r^*) holds for

$$r^* = \inf\left(r > 0 : \ell^*(B \cap rB_2) \leq c_1 r \sqrt{m}\right) \tag{3.4}$$

where $\ell^*(B \cap rB_2)$ is the **Gaussian mean width** of $B \cap rB_2$. We recall the definition of Gaussian mean widths.

Definition 3.8. *Let T be a subset of the Hilbert space H. Denote by $(G_t)_{t \in T}$ the canonical Gaussian process associated with T. The Gaussian mean width of T is*

$$\ell^*(T) = \mathbb{E} \sup_{t \in T} G_t.$$

In finite dimension, the Gaussian process $(G_t)_{t \in T}$ is given by $G_t = \langle G, t \rangle$, where G is a standard Gaussian random variable. For example, in $\mathbb{R}^N$, for every $t = (t_j)_{j=1}^N \in \mathbb{R}^N$, one has $G_t = \sum_{j=1}^N g_i t_i$, where $g_1, \dots, g_N$ are i.i.d. $\mathcal{N}(0,1)$. One then has, for every $T \subset \mathbb{R}^N$,

$$\ell^*(T) = \mathbb{E} \sup_{t \in T} \sum_{j=1}^N g_j t_j.$$

We now introduce a parameter related to the size of the subdifferential of $\|\cdot\|$ at every point.

Definition 3.9. *Let $\|\cdot\|$ be a norm on E, a linear subspace of H. Denote by S the unit sphere of $\|\cdot\|$ and by $\partial \|\cdot\| (x^*)$ its subdifferential at x^*. For every $x^* \in E$, we define the sparsity coefficient of $\|\cdot\|$ at x^* for the radius r^* by*

$$\Delta_{r^*}(x^*) = \inf_{h \in S \cap r^* B_2} \sup_{z \in \partial \|\cdot\|(x^*)} \langle z, h \rangle. \tag{3.5}$$

Since $S \cap r^* B_2 \subset S$ and $\partial \|\cdot\| (x^*) \subset S_*$ when $x^* \neq 0$, one necessarily has, for every $h \in S \cap r^* B_2$ and $z \in \partial \|\cdot\| (x^*)$, $\langle z, h \rangle \leq 1$. The condition of interest below is to require that $\Delta_{r^*}(x^*) > 0$.

Theorem 3.10. *Assume that there exists $r^* > 0$ such that the measurement operator $\mathcal{A}(\cdot)$ satisfies the condition VSR(r^*). Then, for every x^* such that $\Delta_{r^*}(x^*) > 0$, one has*

$$\operatorname*{argmin}_{x : \mathcal{A}(x) = \mathcal{A}(x^*)} \|x\| = \{x^*\}.$$

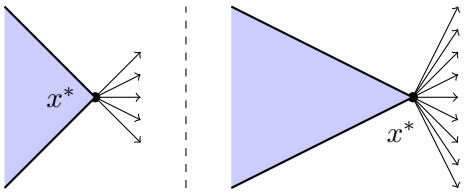


Figure 2: The richness of the subdifferential of $\|\cdot\|$ at the points of interest (that is, points that are sparse in a certain sense)—or in a neighborhood of these points in the noisy case—plays a central role in high-dimensional statistics.

Proof. Let $x^* \in E$ be such that $\Delta_{r^*}(x^*) > 0$. Let $x \in E$ be such that $\mathcal{A}(x) = \mathcal{A}(x^*)$ and $x \neq x^*$. Let us show that $\|x\| > \|x^*\|$.

If $x - x^* \in \mathcal{C}_{r^*}$, then by VSR($r^*$) we have

$$\|\mathcal{A}(x) - \mathcal{A}(x^*)\|_2^2 = \sum_{i=1}^{m} \langle X_i, x^* - x \rangle^2 \geq \frac{m \|x^* - x\|_2^2}{2} > 0$$

because $x \neq x^*$. Then $\mathcal{A}(x) \neq \mathcal{A}(x^*)$, which is a contradiction, and hence $x - x^* \notin \mathcal{C}_{r^*}$. Setting $h = x - x^*$, we then have $\|h\|_2 < r^* \|h\|$, and hence $h/\|h\| \in S \cap r^* B_2$. Then

$$\|x\| - \|x^*\| = \|x^* + h\| - \|x^*\| \geq \langle g, h \rangle = \langle g, \frac{h}{\|h\|} \rangle \|x - x^*\|$$

for every $g \in \partial \|\cdot\| (x^*)$. Passing to the supremum over $g \in \partial \|\cdot\| (x^*)$ and to the infimum over $h/\|h\| \in S \cap r^* \|h\|_2$, we obtain

$$\|x\| - \|x^*\| \geq \Delta_{r^*}(x^*) \|x - x^*\| > 0$$

because $x \neq x^*$ and $\Delta_{r^*}(x^*) > 0$. We therefore indeed have $\|x\| > \|x^*\|$, and hence x^* is the unique solution of the minimization problem

$$\min_{x : \mathcal{A}(x) = \mathcal{A}(x^*)} \|x\| .$$

■

We thus see that the size of the subdifferential of $\|\cdot\|$ at sparse points plays a central role in their reconstruction. To illustrate this point, one may compute:

1. the subgradient of the ℓ_1 norm at s-sparse vectors
2. the subgradient of the nuclear norm at matrices of rank r.

One will then observe that the sparser the vector (respectively, the lower the rank of the matrix), the richer the subdifferential at that point, and hence the more likely Δ_{r^*} is to be strictly positive.

We begin with the computation of the subdifferential of the ℓ_1 norm. As an introduction, one may visualize this subdifferential in dimension $N = 2$, and then we shall study the general case.

The subdifferential of the absolute value function is given by definition, for every $t \in \mathbb{R}$, by

$$\partial |\cdot|(t) = \{g \in \mathbb{R} : |t + h| \geq |t| + gh, \forall h \in \mathbb{R}\}.$$

Let $t > 0$. If $g \in \mathbb{R}$ is a subgradient of the absolute value at t, then for every $h \in \mathbb{R}$ one has $|t + h| \geq |t| + gh$; in particular, for every $-t < h < 0$, $t + h \geq t + gh$, hence $h \geq gh$, and since

$h < 0$ one has $g \geq 1$. Likewise, for every $h > 0$, one has $t + h \geq t + gh$, hence $h \geq gh$, and since $h > 0$ one has $g \leq 1$. Then, necessarily, $g = 1$, and since $g = 1$ is indeed a subgradient of $|\cdot|$ at $t > 0$, because $|t + h| \geq |t| + h$, we indeed have $\partial|\cdot|(t) = \{1\}$. Likewise, when $t < 0$, the unique subgradient of $|\cdot|$ at t is -1. For $t = 0$, g is a subgradient when $|h| \geq gh$ for every $h \in \mathbb{R}$, which is the case for every $g \in [-1, 1]$. We conclude that

$$\partial|\cdot|(t) = \begin{cases} \{1\} & \text{if } t > 0 \\ \{-1\} & \text{if } t < 0 \\ [-1, 1] & \text{if } t = 0. \end{cases}$$

We indeed recover the fact that the subdifferential of a function at a point where it is differentiable reduces to a single point, which is the gradient of that function at that point.

For the computation of the subdifferential of the norm $\|\cdot\|_1$ in dimension 2, consider $x = (x_1, x_2)^\top \in \mathbb{R}^2$. Assume $x_1 \neq 0$, $x_2 \neq 0$. If $g = (g_1, g_2)^\top \in \mathbb{R}^2$ is a subgradient of $\|\cdot\|_1$ at x, then for every $h = (h_1, h_2)^\top \in \mathbb{R}^2$ one must have

$$|x_1 + h_1| + |x_2 + h_2| \geq |x_1| + |x_2| + g_1 h_1 + g_2 h_2.$$

In particular, for $h_1 = 0$ one sees that g_2 is a subgradient of $|\cdot|$ at x_2. Likewise, when $h_2 = 0$, one finds that g_1 is a subgradient of $|\cdot|$ at x_1. We deduce that $g = \operatorname{sign}(x)$. If one of the coordinates of x is zero, then one sees that for that coordinate the subgradient may take any value in $[-1, 1]$. We therefore have, for example,

$$\partial \|\cdot\|_1 \begin{pmatrix} 1/2 \\ 1/2 \end{pmatrix} = \left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right\} \text{ and } \partial \|\cdot\|_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \left\{ \begin{pmatrix} 1 \\ g \end{pmatrix} : |g| \leq 1 \right\}$$

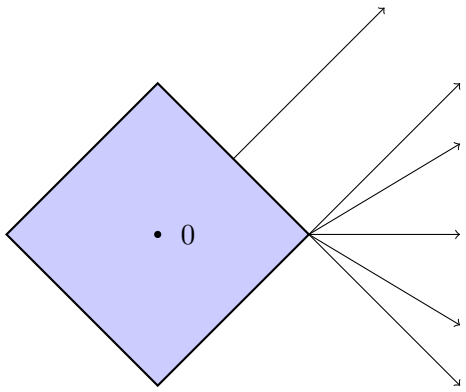


Figure 3: Subdifferential of the norm $\|\cdot\|_1$ in dimension $N = 2$ at $(1/2, 1/2)^\top$ and at $(1, 0)^\top$.

Lemma 3.11. *Let $x^* \in \mathbb{R}^N$ and $1 \leq s \leq N$. Assume that x^* is s-sparse, and write $I = \operatorname{supp}(x^*)$. Then, for $x^* \neq 0$,*

$$\partial \|\cdot\|_1 (x^*) = \{z \in \mathbb{R}^N : \operatorname{sign}(z_i) = x_i^*, \forall i \in I \text{ and } |z_i| \leq 1, \forall i \notin I\}.$$

If $x^* = 0$, then $\partial \|\cdot\|_1 (x^*) = B_\infty^N$.

Proof. We use the characterization of the subdifferential for $x^* \neq 0$:

$$\partial \|\cdot\|_1 (x^*) = \{g \in S_\infty^{N-1} : \langle g, x^* \rangle = \|x^*\|_1\}.$$

One easily sees that g is norming for x^* if and only if $\operatorname{sign}(g_i) = x_i^*$ for every $i \in I$, since

$$\langle g, x^* \rangle = \sum_{i \in I} g_i x_i^* = \sum_{i \in I} |x_i^*| = \|x\|_1$$

only if this condition is satisfied. The coordinates of g on I^c are chosen arbitrarily, subject to $\|g\|_\infty \leq 1$. ∎

In other words, $\partial \|\cdot\|_1 (x^*)$ is isomorphic to a sphere S_∞^{N-s-1} in dimension $\mathbb{R}^{N-s}$. In particular, the sparser x^* is, the richer $\partial \|\cdot\|_1 (x^*)$ is. The extreme cases are given by

$$\partial \|\cdot\|_1 (0) = B_\infty^N, \text{ or } \partial \|\cdot\|_1 ((1, 0, \cdots, 0)) = \{1\} \times S_\infty^{N-2} \text{ and } \partial \|\cdot\|_1 ((1)_{i=1}^N) = \{(1)_{i=1}^N\}.$$

In the 1-sparse case $x^* = (1, 0, \cdots, 0)$, one recovers almost the whole dual sphere S_∞^{N-1} (only the first coordinate is set equal to 1), and in the second case of a *diagonal (or well-spread)* vector, $\|\cdot\|_1$ is differentiable at $(1)_{i=1}^N$, and hence its subdifferential reduces to a single point, which is the gradient of the norm $\|\cdot\|_1$ at that point.

One can obtain a similar result for the subdifferential of the nuclear norm at a low-rank matrix. The notions of support and of sign can be adapted to the matrix case.

Theorem 3.12. *Let $A \in \mathbb{R}^{u \times v}$ with $u \geq v$. Consider an SVD of A: $A = UDV^\top$, where $D = \operatorname{diag}(\sigma_A) \in \mathbb{R}^{u \times v}$ and $\sigma_A = (\sigma_1, \ldots, \sigma_{u \wedge v})$ is the spectrum of A, with $\sigma_1 \geq \ldots \geq \sigma_{u \wedge v}(A)$. Assume that A has rank r. Consider the block decomposition*

$$U = \begin{bmatrix} U^{(1)} & U^{(2)} \end{bmatrix} \in \mathcal{O}(u), D = \begin{bmatrix} \operatorname{diag}(\sigma_1, \ldots, \sigma_r) & 0_{r,v-r} \\ 0_{u-r,r} & 0_{u-r,v-r} \end{bmatrix} \text{ and } V = \begin{bmatrix} V^{(1)} & V^{(2)} \end{bmatrix} \in \mathcal{O}(v)$$

where $U^{(1)} \in \mathbb{R}^{u \times r}$, $V^{(1)} \in \mathbb{R}^{v \times r}$. The subdifferential of $\|\cdot\|_{S_1}$ at A is

$$\partial \|\cdot\|_{S_1} (A) = \left\{ U^{(1)} {V^{(1)}}^\top + U^{(2)} W {V^{(2)}}^\top : W \in \mathbb{R}^{u-r, v-r}, \|W\|_{S_\infty} \leq 1 \right\}$$

To draw the analogy with the subdifferential of the ℓ_1 norm, one may say that the term $U^{(1)} {V^{(1)}}^\top$ plays the role of the "sign" of A, whereas the term $U^{(2)} W {V^{(2)}}^\top$ is the term (which induces the richness of the subdifferential) corresponding to matrices whose "support" differs from that of A: one has, for every $W \in \mathbb{R}^{u-r, v-r}$, $\left\langle A, U^{(2)} W {V^{(2)}}^\top \right\rangle = 0$. One can see the analogy by rewriting the matrix A in the basis $(u_i v_j^\top)_{i,j}$ associated with an SVD of A:

$$A = UDV^\top = \sum_i \sigma_i u_i v_i^\top$$

where the u_i and v_i are the columns of U and V, respectively. Thus, viewing $(u_i v_i^\top)_i$ as an orthogonal family of rank-1 matrices in which A is represented naturally (one notes that $\langle A, u_i v_j^\top \rangle = 0$ when $i \neq j$ and $\langle A, u_i v_i^\top \rangle = \sigma_i$), one may write every subgradient of $\|\cdot\|_{S_1}$ at A as

$$\sum_{i=1}^r u_i v_i^\top + U^{(2)} W {V^{(2)}}^\top$$

for $W \in \mathbb{R}^{u-r, v-r}$ such that $\|W\|_{S_\infty} \leq 1$, where "$U^{(2)} W {V^{(2)}}^\top$" is viewed as a term whose support differs from that of A.

Before proving Theorem 3.12, we introduce the following notion: a norm $\|\cdot\|$ on $\mathbb{R}^{u \times v}$ is said to be **orthogonally invariant** when, for every $A \in \mathbb{R}^{u \times v}$, $U \in \mathcal{O}(u)$, and $V \in \mathcal{O}(v)$, one has

$\|UAV\| = \|A\|$. In that case, one can show that there exists an unconditional norm $\phi(\cdot)$ on $\mathbb{R}^{u\wedge v}$ such that $\|A\| = \phi(\sigma_A)$, where $\sigma_A \in \mathbb{R}^{u\times v}$ is the spectrum of A. We recall that a norm is **unconditional** when

$$\phi(\epsilon_1 x_1, \dots, \epsilon_{u\wedge v} x_{u\wedge v}) = \phi(x_1, \dots, x_{u\wedge v})$$

for all signs $\epsilon_1, \dots, \epsilon_{u\wedge v} \in \{\pm 1\}$.

To prove Theorem 3.12, we shall use the following result on the directional derivative of an orthogonally invariant norm:

$$\lim_{\epsilon\to 0^+} \frac{\|A+\epsilon R\| - \|A\|}{\epsilon} = \max_{d\in\partial\phi(\sigma_A)} \sum_{i=1}^{u\wedge v} d_i u_i v_i^\top \tag{3.6}$$

where $A = UDV^\top$, $U = [u_1|\cdots|u_u]$, $V = [v_1|\cdots|v_v]$, and $D = \operatorname{diag}(\sigma_A)$. We recall that one always has

$$\lim_{\epsilon\to 0^+} \frac{\|A+\epsilon R\| - \|A\|}{\epsilon} = \max_{G\in\partial\|\cdot\|(A)} \big\langle R, G\big\rangle. \tag{3.7}$$

We now prove the following lemma.

Lemma 3.13. *Let $\|\cdot\|$ be an orthogonally invariant norm on $\mathbb{R}^{u\times v}$. Denote by $\phi(\cdot)$ an unconditional norm on $\mathbb{R}^{u\wedge v}$ such that, for every $A \in \mathbb{R}^{u\times v}$, $\|A\| = \phi(\sigma_A)$, where σ_A is the spectrum of A. Let $A \in \mathbb{R}^{u\times v}$. Then*

$$\partial\,\|\cdot\|\,(A) = \operatorname{conv}\big\{U\Sigma V^\top : A = UDV^\top, \Sigma = \operatorname{diag}(\sigma), \sigma \in \partial\phi(\sigma_A)\big\}$$

where all SVDs of A are to be considered in the construction of the set on the right-hand side.

Proof. Write $S(A) = \{U\Sigma V^\top : A = UDV^\top, \Sigma = \operatorname{diag}(\sigma), \sigma \in \partial\phi(\sigma_A)\}$. Let $G \in \operatorname{conv}(S(A))$. Let us show that G is a subgradient of $\|\cdot\|$ at A. We recall that if $A \neq 0$, then

$$\partial\,\|\cdot\|\,(A) = \big\{G \in S_* : \big\langle G, A\big\rangle = \|A\|\,\big\}$$

where S_* is the unit sphere of the dual norm of $\|\cdot\|$. It therefore suffices to show that $\big\langle G, A\big\rangle = \|A\|$ and that, for every R such that $\|R\| \le 1$, one has $\big\langle G, R\big\rangle \le 1$.

We write $G = \sum_i \lambda_i U_i \Sigma_i V_i^\top$, where $\lambda_i \ge 0$, $\sum_i \lambda_i = 1$, $\Sigma_i = \operatorname{diag}(\sigma_i)$ with $\sigma_i \in \partial\phi(\sigma_A)$, and $A = U_i D V_i^\top$ is an SVD of A. We have

$$\big\langle G, A\big\rangle = \sum_i \lambda_i \big\langle U_i\Sigma_i V_i^\top, U_i D V_i^\top\big\rangle = \sum_i \lambda_i \big\langle \sigma_i, \sigma_A\big\rangle = \sum_i \lambda_i \phi(\sigma_A) = \phi(\sigma_A) = \|A\|$$

because $\big\langle \sigma_i, \sigma_A\big\rangle = \phi(\sigma_A)$, since $\sigma_i \in \partial\phi(\sigma_A)$.

For the second point, take $R \in \mathbb{R}^{u\times v}$ such that $\|R\| \le 1$. Consider σ_R, the spectrum of R. By von Neumann's trace inequality,

$$\big\langle G, R\big\rangle = \sum_i \lambda_i \big\langle U_i\Sigma_i V_i^\top, R\big\rangle \le \sum_i \lambda_i \big\langle \sigma_i, \sigma_R\big\rangle \le 1$$

because $\sigma_i \in S_*$ and $\sigma_R \in B$, the unit ball of $\|\cdot\|$, hence $\big\langle \sigma_i, \sigma_R\big\rangle \le 1$. We therefore deduce that $G \in \partial\,\|\cdot\|\,(A)$.

Conversely, assume that there exists $G \in \partial\,\|\cdot\|\,(A)$ with $G \notin \operatorname{conv}(S(A))$. By a separation argument for convex sets, there exists R such that $\big\langle R, H\big\rangle < \big\langle R, G\big\rangle$ for every $H \in S(A)$. We therefore have

$$\max_{H\in S(A)} \big\langle R, H\big\rangle < \max_{G\in\partial\|\cdot\|(A)} \big\langle R, G\big\rangle$$

and then

$$\max_{\sigma\in\partial\phi(\sigma_A)} \sum_i \sigma_i u_i^\top R v_i < \max_{G\in\partial\|\cdot\|(A)} \langle R, G\rangle.$$

This contradicts the identities (3.6) and (3.7). ∎

Proof of Theorem 3.12 We apply Lemma 3.13 in the case $\phi(\sigma) = \|\sigma\|_1$. Let A have rank r, and consider an SVD $A = UDV^\top$ with the block decomposition

$$U = \begin{bmatrix} U^{(1)} & U^{(2)} \end{bmatrix} \in \mathcal{O}(u), D = \begin{bmatrix} \operatorname{diag}(\sigma_1,\dots,\sigma_r) & 0_{r,v-r} \\ 0_{u-r,r} & 0_{u-r,v-r} \end{bmatrix} \text{ and } V = \begin{bmatrix} V^{(1)} & V^{(2)} \end{bmatrix} \in \mathcal{O}(v)$$

where $U^{(1)} \in \mathbb{R}^{u\times r}$, $V^{(1)} \in \mathbb{R}^{v\times r}$.

Let $G \in \partial\|\cdot\|_{S_1}(A)$. By Lemma 3.13, there exist $\lambda_i \geq 0$ with $\sum_i \lambda_i = 1$, and SVDs $A = U_i D V_i^\top$ of A, such that

$$G = \sum_i \lambda_i U_i \Sigma_i V_i^\top$$

where $\Sigma_i = \operatorname{diag}(\sigma_i)$ and $\sigma_i \in \partial\|\cdot\|_1(\sigma_A)$. Since

$$\partial\|\cdot\|_1(\sigma_A) = \{g \in \mathbb{R}^{u\wedge v} : g_i = 1, i = 1,\dots,r \text{ and } |g_i| \leq 1, i \geq r+1\},$$

if $(U_i)_j$ and $(V_i)_j$ are the columns of U_i and V_i, then

$$G = \sum_i \lambda_i \Big(\sum_j \sigma_{ij} (U_i)_j (V_i)_j^\top \Big) = U^{(1)} {V^{(1)}}^\top + \sum_i \lambda_i U_i^{(2)} W_i {V_i^{(2)}}^\top$$

where the W_i are diagonal matrices of size $(u-r, v-r)$ whose diagonal entries lie in $[-1,1]$. Moreover, for every i, one can find orthogonal matrices $Y_i \in \mathcal{O}(u-r)$ and $Z_i \in \mathcal{O}(v-r)$ such that $U_i^{(2)} = U^{(2)} Y_i$ and $V_i^{(2)} = V^{(2)} Z_i$. We therefore have

$$G = U^{(1)} {V^{(1)}}^\top + \sum_i \lambda_i U^{(2)} Y_i W_i Z_i^\top {V^{(2)}}^\top = U^{(1)} {V^{(1)}}^\top + U^{(2)} W {V^{(2)}}^\top$$

where $W = \sum_i \lambda_i Y_i W_i Z_i^\top$. Moreover, the largest singular value of W satisfies

$$\sigma_1(W) = \sigma_1\Big(\sum_i \lambda_i Y_i W_i Z_i^\top \Big) \leq \sum_i \lambda_i \sigma_1(Y_i W_i Z_i \top) = \sum_i \lambda_i \sigma_1(W_i) \leq 1.$$

∎

Another way of characterizing the subdifferential $\partial\|\cdot\|_{S_1}(A)$, so as to bring out more clearly the notion of "support" associated with a matrix, is to write it in the following form. For $A \in \mathbb{R}^{u\times v}$, one can see that A has rank r if and only if there exist two linear subspaces $I \subset \mathbb{R}^u$ and $J \subset \mathbb{R}^v$ of dimension r such that $A = P_I A P_J$, where P_I is the projection onto I and P_J is the projection onto J. In that case, $\partial\|\cdot\|_{S_1}(A)$ is the set of all $G \in \mathbb{R}^{u\times v}$ such that, for every $B \in \mathbb{R}^{u\times v}$,

1. $\langle G, A\rangle = \|A\|_{S_1}$ — that is, G is norming for A
2. $\langle G, P_{I^\perp} B P_{J^\perp}\rangle = \|P_{I^\perp} B P_{J^\perp}\|_{S_1}$ — that is, G is norming for every matrix whose "support" is disjoint from that of A
3. $\langle G, P_I B P_{J^\perp}\rangle = 0$ and $\langle G, P_{I^\perp} B P_J\rangle = 0$ — that is, the "support" of G is restricted to "$(I,J) \cup (I^\perp, J^\perp)$".

One can also characterize every subgradient in $\partial \|\cdot\|_{S_1}(A)$ by writing it in the basis $(u_i v_j^\top)_{i,j}$ of the SVD of $A = UDV^\top$, where the u_i and v_j are the columns of U and V. Indeed, $G \in \partial \|\cdot\|_{S_1}(A)$ if and only if

$$G = \sum_{i=1}^{r} u_i v_i^\top + \sum_{i=1}^{u \wedge v} u_i W v_i^\top \tag{3.8}$$

where $\|W\|_{S_\infty} \leq 1$.

Taking any expression for the subgradients of $\|\cdot\|_{S_1}$ at a matrix A^* of rank r, one sees that the smaller r is, the richer the subdifferential of S_1 at A^* is. This is the property one seeks in order to have $\Delta_{r^*}(A^*) > 0$.

Conclusion : The subdifferential of a function is larger at its points of nondifferentiability. This is the case for the norms $\|\cdot\|_1$ and $\|\cdot\|_{S_1}$ at sparse vectors and at low-rank matrices. It is an essential property of these norms for the Basis Pursuit and Nuclear Norm Minimization procedures, which seek to reconstruct this type of object. The essential idea in the construction of regularization norms that induce sparsity is to create singularities at the objects that are the sparsest for the desired notion of sparsity.

4 Proximal gradient descent for matrix completion

We saw in the previous sections that the nuclear norm minimization procedure can be rewritten as an SDP optimization problem in Theorem 2.7, and can therefore be solved numerically (approximately, and for reasonable dimensions). In particular, this approach provides a first numerical solution to the matrix completion problem, and can therefore be used for the design of recommender systems. In this section, we propose other algorithmic solutions to the matrix completion problem, which have the advantage of avoiding an SDP reformulation (which can be difficult to solve in high dimension).

We now introduce two gradient descent algorithms. In the first case, we speak of *projected gradient descent*, and in the second case of *proximal gradient descent*. We consider here the measurements obtained in this problem, that is, each observation is the datum of an entry of the matrix to be reconstructed. One may then speak of a **mask** that reveals only part of the matrix:

$$P_\Omega : \left\{ \begin{array}{ccc} \mathbb{R}^{u \times v} & \to & \mathbb{R}^{u \times v} \\ A & \to & P_\Omega(A) \end{array} \right. \quad \text{where } (P_\Omega(A))_{i,j} = \left\{ \begin{array}{cc} A_{i,j} & \text{if } (i,j) \in \Omega \\ 0 & \text{otherwise} \end{array} \right.$$

where $\Omega \subset \{1, \ldots, u\} \times \{1, \ldots, v\}$ is the set of indices of the revealed entries. One sees in particular that P_Ω is a projection operator (i.e., $P_\Omega^2 = P_\Omega$) and that it is self-adjoint (i.e., $P_\Omega^\top = P_\Omega$).

For the matrix completion problem, one can write these observations in condensed form: one observes $P_\Omega(A^*)$ for some set Ω and some matrix A^* to be reconstructed, that is, to be completed (for example, the users/items matrix). We assume that A^* has low rank. This is the central "low-dimensional" assumption for this problem.

One may then consider two other procedures, in addition to *rank minimization* and its convex relaxation given by *nuclear norm minimization*:

1.

$$\tilde{A} \in \underset{\text{rang}(A) \leq r}{\text{argmin}} \|P_\Omega(A^*) - P_\Omega(A)\|_{S_2} \tag{4.1}$$

2.

$$\bar{A} \in \operatorname*{argmin}_{A} \left(\frac{1}{2} \left\| P_\Omega(A^*) - P_\Omega(A) \right\|_{S_2}^2 + \lambda \left\| A \right\|_{S_1} \right). \tag{4.2}$$

In both cases, one sees a data-fidelity term $A \to \left\| P_\Omega(A^*) - P_\Omega(A) \right\|_{S_2}$, together with a rank constraint in the first case and a regularization term given by the nuclear norm in the second case.

To implement these two procedures, we shall consider matrix analogues of the vector functions known as "*hard-thresholding*" and "*soft-thresholding*". In our matrix case, we shall see that these procedures act on the spectrum of the matrix. We begin by describing an algorithm for the implementation of an (approximate) solution to (4.1).

A projected gradient algorithm for (4.1). A classical approach to the numerical solution of a constrained optimization problem is to use a projected gradient descent method. To this end, one must be able to compute the gradient of the objective function and to project onto the constraint. The canonical setting for this algorithmic solution is to require a convex and differentiable objective function and a convex constraint. For problem (4.1), the objective function is differentiable and its gradient is easily computed (one notes that minimizing $A \to \left\| P_\Omega(A^*) - P_\Omega(A) \right\|_{S_2}$ or $A \to (1/2) \left\| P_\Omega(A^*) - P_\Omega(A) \right\|_{S_2}^2$ amounts to the same thing here): for $F : A \in \mathbb{R}^{u \times v} \to (1/2) \left\| P_\Omega(A^*) - P_\Omega(A) \right\|_{S_2}^2$, one has

$$\nabla F(A) = -P_\Omega^\top (P_\Omega(A^*) - P_\Omega(A)) = P_\Omega(A) - P_\Omega(A^*) \tag{4.3}$$

because $P_\Omega^2 = P_\Omega$ and $P_\Omega^\top = P_\Omega$.

By contrast, the constraint in (4.1) is the set of matrices of rank at most r; it is therefore not a convex constraint and, in general, it is difficult to project onto a set that is not convex. However, it turns out that in the particular case of the set of matrices of rank at most r, one does know how to project onto this set. This is the object of the following proposition. We first recall that the set of projections of A onto $\Sigma_r = \{B : \operatorname{rang}(B) \leq r\}$ is given by

$$\operatorname*{argmin}_{B \in \Sigma_r} \left\| A - B \right\|_{S_2} \tag{4.4}$$

We show in the following result that a projection is obtained by computing the SVD of A and then hard-thresholding the spectrum vector of A; this projection is unique when $\sigma_r > \sigma_{r+1}$.

Proposition 4.1. *Let $A \in \mathbb{R}^{u \times v}$. Consider the singular value decomposition of A given by $A = PDQ^\top$, where $P \in \mathcal{O}(u)$, $Q \in \mathcal{O}(v)$, and $D = \operatorname{diag}(\sigma_1, \cdots, \sigma_{u \wedge v}) \in \mathbb{R}^{u \times v}$, where $\sigma_1 \geq \cdots \geq \sigma_{u \wedge v} \geq 0$ are the singular values of A. A projection of A onto the set of matrices of rank at most r is $PD_rQ^\top$, where D_r is the $u \times v$ diagonal matrix $\operatorname{diag}(\sigma_1, \cdots, \sigma_r, 0, \cdots, 0)$, and it is the unique projection when $\sigma_r > \sigma_{r+1}$. In other words, one has*

$$PD_rQ^\top \in \operatorname*{argmin}_{B : \operatorname{rang}(B) \leq r} \left\| A - B \right\|_{S_2}.$$

Proof. We know from von Neumann's inequality (see Proposition 2.6) that for every $B \in \mathbb{R}^{u \times v}$,

$$\left\| A - B \right\|_{S_2} \geq \left\| \sigma_A - \sigma_B \right\|_2 \tag{4.5}$$

where σ_A (resp. σ_B) is the spectrum of A (resp. B), that is, the vector of size $u \wedge v$ of the singular values of A (resp. of B) arranged in nonincreasing order. Moreover, equality holds in (4.5) if and only if B is of the form $B = P\operatorname{diag}(\sigma_B)Q^\top$. The set of solutions to (4.4) is therefore a subset of the set of matrices of the form $B = P\operatorname{diag}(\sigma_B)Q^\top$. It then remains to optimize over the spectrum σ_B. One sees that if B has rank r, then $\left\| \sigma_A - \sigma_B \right\|_2^2 \geq \sum_{j>r} \sigma_j(A)^2$, and this value is attained for $\sigma_B = (\sigma_1, \cdots, \sigma_r, 0, \cdots, 0)^\top$. ∎

We thus see that in order to project a matrix A onto the set of matrices of rank at most r, it suffices to be able to compute its SVD and to threshold its spectrum by keeping only its r largest singular values. There are algorithms that compute the SVD of a $u \times v$ matrix in $\mathcal{O}(\min(u^2 v, v^2 u))$, and even in our case, since one need only compute the first r singular vectors and singular values, one can do so in $\mathcal{O}(r^2 \min(u, v))$. One therefore knows how to project onto $\{B : \operatorname{rang}(B) \leq r\}$ for dimensions that are not too large.

We thus have the two ingredients for the construction of a projected gradient algorithm associated with problem (4.1): the gradient of the objective function and the projection operator onto the constraint. Given a sequence of *step sizes* $(\gamma_k)_k$ (a decreasing sequence in $\mathbb{R}_+^*$), one obtains the following projected gradient descent algorithm

$$A^{k+1} = S_r^{hard}\big(A^k - \gamma_k \nabla F(A^k))\big) = S_r^{hard}\big(A^k + \gamma_k (P_\Omega(A^*) - P_\Omega(A^k))\big) \tag{4.6}$$

where S_r^{hard} is the hard spectral thresholding operator at rank r: if $A = PDQ^\top$, where $P \in \mathcal{O}(u)$, $Q \in \mathcal{O}(v)$, and $D = \operatorname{diag}(\sigma_1, \cdots, \sigma_{u \wedge v}) \in \mathbb{R}^{u \times v}$, then

$$S_r^{hard}(A) = P D_r^{hard} Q^\top \text{ where } D_r^{hard} = \operatorname{diag}(\sigma_1, \sigma_2, \ldots, \sigma_r, 0, \cdots, 0).$$

Along the way, one sees that if one takes $\gamma_k = 1$ for every $k \in \mathbb{N}$ in (4.6), then $A^{k+1} = S_r^{hard}\big(P_\Omega(A^*) + P_{\Omega^c}(A^k))\big)$, where Ω^c is the complement of Ω. Thus, the algorithm is obtained by replacing, at each iteration, the entries of A^k indexed by Ω by those of A^* (that is, by setting $P_\Omega A^k \leftarrow P_\Omega A^*$) and by performing a rank-r PCA on the resulting matrix. One obtains A^{k+1} and starts again. One recognizes an algorithm used for *missing data*, called ACPmiss or missMDA. This algorithm therefore provides an (algorithmic) solution to the missing-data problem based on the low-rank assumption.

Proximal gradient descent for (4.2). For the implementation of (4.2), we shall construct a proximal gradient descent algorithm. This type of algorithm may be viewed as a generalization of projected gradient descent. We begin with a few reminders about this algorithm.

We recall that a convex function $f : U \to \mathbb{R}$ is not necessarily differentiable at every point of its domain $U \subset \mathbb{R}^n$, a convex set (one knows that it is differentiable almost everywhere). However, at every point $x \in U$ it admits a **subdifferential**, which is the set

$$\partial^- f(x) = \left\{ g \in \mathbb{R}^n : f(y) \geq f(x) + \langle g, y - x \rangle, \forall y \in U \right\}. \tag{4.7}$$

An element of the subdifferential $\partial^- f(x)$ is called a **subgradient**. We have already seen computations of subgradients in the previous sections. The other notion we shall need is that of a **proximal operator**. To introduce this notion, one may view it as a generalization of projection operators. Indeed, if $K \subset \mathbb{R}^n$ and $i_K : x \in \mathbb{R}^n \to 0$ if $x \in K$ and $+\infty$ if $x \notin K$, then one sees that the set of projections of $x \in \mathbb{R}^n$ onto K satisfies

$$\operatorname*{argmin}_{y \in K} \|x - y\|_2 = \operatorname*{argmin}_{y \in \mathbb{R}^n} \left(\frac{1}{2} \|x - y\|_2^2 + i_K(y) \right). \tag{4.8}$$

One may then generalize this notion to any convex function, and not only to indicator functions of sets such as i_K. Indeed, if $g : \mathbb{R}^n \to \mathbb{R}$ is a convex function, the proximal operator of g is the unique solution

$$\operatorname{prox}_g : x \in \mathbb{R}^n \to \operatorname*{argmin}_{u \in \mathbb{R}^n} \left(\frac{1}{2} \|u - x\|_2^2 + g(u) \right). \tag{4.9}$$

In what follows we shall compute the proximal operator of the S_1 norm in order to construct a proximal gradient descent algorithm; it is based on the proximal operator of the ℓ_1 norm, which we now give.

Example 4.2. *[**Proximal mapping of the** ℓ_1 **norm and of** ℓ_0] The proximal operator of $\ell_1 : x \to \|x\|_1$ is the soft-thresholding operator. Indeed, one sees that the function to be optimized that defines the proximal operator of $\gamma\ell_1$ (for $\gamma > 0$) is separable: for every $u \in \mathbb{R}^N$,*

$$\frac{1}{2}\|u - z\|_2^2 + \gamma \|u\|_1 = \sum_{j=1}^{N} \Big(\frac{1}{2}(u_j - z_j)^2 + \gamma|u_j|\Big).$$

It then suffices to minimize each term of this sum. One is therefore led to find a solution of the (scalar) problem: for every $s \in \mathbb{R}$,

$$\operatorname*{argmin}_{t\in\mathbb{R}} \Big(\frac{1}{2}(s-t)^2 + \gamma|t|\Big).$$

To obtain this minimum, one studies $\varphi(t) = (s-t)^2/2 + \gamma|t|$ on $(-\infty, 0)$, $\{0\}$, and $(0, +\infty)$. One sees that φ attains its minimum at $s - \gamma$ on $(0, \infty)$ when $s - \gamma \geq 0$, and that this minimum is smaller than $\varphi(0)$ under this condition. Likewise, when $s \leq -\gamma$, φ attains its minimum at $s + \gamma$ on $(-\infty, 0)$ and is smaller than at zero. Finally, when $-\gamma \leq s \leq \gamma$, one sees that the minimum of φ is attained at 0. We therefore have

$$\operatorname*{argmin}_{t\in\mathbb{R}} \Big(\frac{1}{2}(s-t)^2 + \gamma|t|\Big) = \begin{cases} s-\gamma & \text{if } s \geq \gamma \\ 0 & \text{if } -\gamma \leq s \leq \gamma \\ s+\gamma & \text{if } s \leq -\gamma \end{cases} = \operatorname{sgn}(s)\big(|s| - \gamma\big)_+$$

which is the soft-thresholding function for a threshold equal to γ. The hard-thresholding function is $s \to sI(|s| \geq \gamma)$, which can be obtained as the proximal operator of the indicator of 0:

$$\operatorname*{argmin}_{t\in\mathbb{R}} \Big(\frac{1}{2}(s-t)^2 + \frac{\gamma^2}{2} I(t \neq 0)\Big) = sI(|s| \geq \gamma)$$

where $I(t \neq 0)$ equals 1 when $t \neq 0$ and 0 otherwise. We therefore have, for every $z \in \mathbb{R}^N$,

$$\boxed{\operatorname{prox}_{\gamma\|\cdot\|_1}(z) = \big(\operatorname{sgn}(z_j)(|z_j| - \gamma)_+\big)_{j=1}^N} \tag{4.10}$$

which is the coordinatewise soft-thresholding function, and

$$\boxed{\operatorname{prox}_{(\gamma^2/2)\|\cdot\|_0}(z) = \big(z_j I(|z_j| \geq \gamma)\big)_{j=1}^N} \tag{4.11}$$

which is the coordinatewise hard-thresholding function.

The proximal operator has several interesting properties, such as that of being contractive. The property of interest here is the following, because it justifies proximal methods.

Proposition 4.3. *Let $f : \mathbb{R}^n \to \mathbb{R}$ be a differentiable convex function and $g : \mathbb{R}^n \to \mathbb{R}$ a convex function. Let $\gamma > 0$. The following two assertions are equivalent:*

a) x^* is a solution of $\min_{x\in\mathbb{R}^n} (f(x) + g(x))$

b) $x^* = \operatorname{prox}_{\gamma g}(x^* - \gamma\nabla f(x^*))$.

Proof. Replacing $f+g$ by $\gamma f+\gamma g$ in *a)*, one sees that one may take $\gamma=1$. We first show that for every $z\in\mathbb{R}^n$, one has the equivalence

$$u^*=\mathrm{prox}_g(z) \text{ if and only if } z-u^*\in\partial^- g(u^*). \tag{4.12}$$

Indeed, by definition of the proximal operator, one sees that $u^*=\mathrm{prox}_g(z)$ if and only if $0\in(u^*-z)+\partial^- g(u^*)$, and hence (4.12) does hold. We therefore have the following chain of equivalences:

$$x^*=\mathrm{prox}_g\left(x^*-\nabla f(x^*)\right)\Leftrightarrow x^*-\nabla f(x^*)-x^*\in\partial^- g(x^*)\Leftrightarrow 0\in\nabla f(x^*)+\partial^- g(x^*).$$

This proves the result. ∎

Proposition 4.3 shows that, in order to find a solution of a problem of minimizing the sum of two convex functions, at least one of which is differentiable (the other need not be), it suffices to find a fixed point of the function $x\to\mathrm{prox}_{\gamma g}\left(x-\gamma\nabla f(x)\right)$. Proposition 4.3 therefore links a convex optimization problem to a fixed-point problem. In particular, one can construct a fixed-point algorithm to solve a problem of the type $\min_{x\in\mathbb{R}^n}\left(f(x)+g(x)\right)$. In our case, in order to find $x^*=G(x^*)$ where $G:x\to\mathrm{prox}_{\gamma g}\left(x-\gamma\nabla f(x)\right)$, this algorithm is written $x_{k+1}=G(x_k)$, and one knows that it converges to some $x^*=G(x^*)$, for example when G is a contraction (i.e., $|G(u)-G(v)|<\|u-v\|_2$, $\forall u,v\in\mathbb{R}^n$). This is the algorithm that we shall also develop here, except that at each iteration one changes the value of γ. One obtains the following algorithm

$$A_{k+1}=\mathrm{prox}_{\lambda\gamma_k\|\cdot\|_{S_1}}\Big(A_k+\gamma_k(P_\Omega(A^*)-P_\Omega(A_k))\Big)$$

where we have used the formula for the gradient of F given in (4.3). The last thing that remains to be done in order to actually run this algorithm is to compute the proximal operator of $\gamma\|\cdot\|_{S_1}$. This is what we do now.

Proposition 4.4. *Let $\gamma>0$. Let $A\in\mathbb{R}^{u\times v}$, and write $A=UDV^\top$ for the SVD of A. The proximal operator of $\gamma\|\cdot\|_{S_1}$ at A is the soft spectral thresholding operator:*

$$\mathrm{prox}_{\gamma\|\cdot\|_{S_1}}(A)=S_\gamma^{soft}(A)=UD_\gamma V^\top$$

where $D_\gamma=\mathrm{diag}((\sigma_1-\gamma)_+,(\sigma_2-\gamma)_+,\cdots,(\sigma_{u\wedge v}-\gamma)_+)$ is the $u\times v$ diagonal matrix whose diagonal entries are obtained by applying the soft-thresholding function at level γ to the singular values of A.

Proof. By definition,

$$\mathrm{prox}_{\gamma\|\cdot\|_{S_1}}(A)\in\operatorname*{argmin}_{B\in\mathbb{R}^{u\times v}}\Big(\frac{1}{2}\|A-B\|_{S_2}^2+\gamma\|B\|_{S_1}\Big).$$

By von Neumann's inequality, one shows that the minimum is attained at a point B of the form $B=UD_BV^\top$, where $D_B=\mathrm{diag}(\sigma_B)$ and σ_B is the spectrum of B. Indeed, by von Neumann, one has

$$\|A-B\|_{S_2}^2=\|A\|_{S_2}^2+\|B\|_{S_2}^2-2\langle A,B\rangle\geq\|A\|_{S_2}^2+\|B\|_{S_2}^2-2\langle\sigma_A,\sigma_B\rangle=\|\sigma_A-\sigma_B\|_2^2.$$

We therefore have $\mathrm{prox}_{\gamma\|\cdot\|_{S_1}}(A)=U\mathrm{diag}(\hat{d})V^\top$, where

$$\hat{d}\in\operatorname*{argmin}_{\sigma\in\mathbb{R}^{u\wedge v}}\Big(\frac{1}{2}\|\sigma_A-\sigma\|_2^2+\gamma\|\sigma\|_1\Big)=\mathrm{prox}_{\gamma\|\cdot\|_1}(\sigma_A).$$

Using the expression for the proximal mapping of the norm $\|\cdot\|_1$, we therefore recover the soft-thresholding operator applied to σ_A (which has all of its coordinates nonnegative and ordered), and hence

$$\mathrm{prox}_{\gamma\|\cdot\|_1}(\sigma_A) = \big((\sigma_1-\gamma)_+, (\sigma_2-\gamma)_+, \ldots, (\sigma_{u\wedge v}-\gamma)_+\big)^\top.$$

We indeed obtain the result. ∎

Once the proximal operator of $\gamma\,\|\cdot\|_{S_1}$ has been computed in Proposition 4.4, one can implement the proximal gradient descent method for (4.2):

$$A_{k+1} = S^{soft}_{\lambda\gamma_k}\Big(A_k - \gamma_k\big(P_\Omega(A_k) - P_\Omega(A^*)\big)\Big) \tag{4.13}$$

where $S^{soft}_{\lambda\gamma_k}$ is the soft spectral thresholding function at level $\lambda\gamma_k$ and $(\gamma_k)_k$ is a family of step sizes.

One thus sees that the only difference between the projected gradient descent algorithm associated with (2.1) obtained in (4.6) and the proximal gradient descent algorithm obtained in (4.13) for approximating a solution of (4.2) is that, instead of hard thresholding, one performs soft thresholding. The two algorithms therefore have the same computational cost. Both are implemented without resorting to SDPs, but they require computing SVDs (or thresholded SVDs) at each iteration, which can be expensive.

Key point : One can project onto the space of low-rank matrices and compute the proximal operator of the nuclear norm. One can then construct a matrix completion algorithm by a projected gradient method or a proximal gradient method.

5 Matrix factorization algorithms and incorporation of metadata for recommender systems.

The idea of the matrix factorization approach is to regard the space $\mathbb{R}^d$ as a latent space in which one represents both the *users* and the *items* by vectors $(p_i)_{1\le i\le u}$ and $(q_j)_{1\le j\le v}$, where

1. p_i is the representation of user i in the latent space $\mathbb{R}^d$
2. q_j is the representation of item j in the latent space $\mathbb{R}^d$

such that the inner product $\langle p_i, q_j\rangle_{\mathbb{R}^d}$ measures the affinity between *item* j and *user* i.

For example, a latent space for the Netflix data is represented in Figure 4.

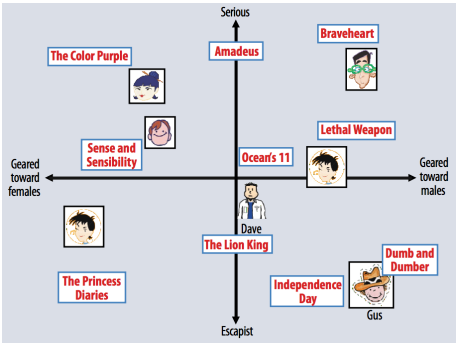


Figure 4: Latent space, here $\mathbb{R}^2$, in which users and items (here, movies, in the case of the Netflix data) are represented in the same place.

Factorization of the users/items matrix for the design of a recommender system. One has item-rating data by users of the form $\{(user_i, item_j, grade_{ij}) : (i,j) \in \Omega\}$, where $\Omega \subset \{1,\dots,u\} \times \{1,\dots,v\}$. One seeks to construct the two families of vectors $(p_i)_{1\le i\le u}$ and $(q_j)_{1\le j\le v}$ representing the users and the items in the latent space $\mathbb{R}^d$.

The idea is to construct $(p_i)_{1\le i\le u}$ and $(q_j)_{1\le j\le v}$ so that the similarity measures $\langle p_i, q_j\rangle$ agree as closely as possible with the observed ratings, that is, one seeks to have

$$\langle p_i, q_j\rangle \approx grade_{ij} \quad \forall (i,j) \in \Omega$$

All the other values $\langle p_i, q_j\rangle$ for $(i,j) \notin \Omega$ are then used to complete the users/items matrix. The matrix factorization problem for the design of a recommender system then amounts to writing the matrix A^* as the product of two matrices, as represented in Figure 5.

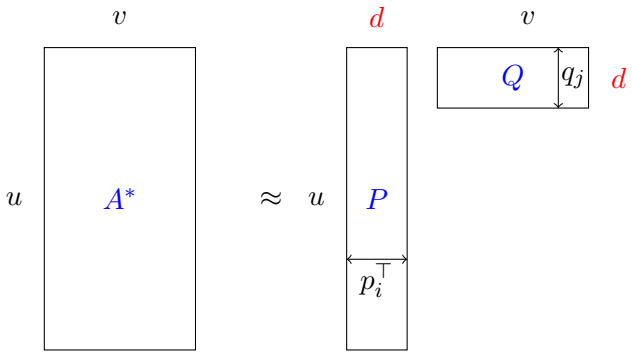


Figure 5: Matrix representation of the problem of factorizing the matrix A^* as the product PQ.

One sees in particular that the low-rank assumption appears here explicitly, since the product PQ has rank at most d. Thus, by seeking to approximate A^* by the product PQ, one is in particular approximating A^* by a low-rank matrix. This approximation has a chance of being good only if A^* is indeed approximately low rank.

As indicated above, one knows only a few entries of the matrix A^*. It is on these entries that we shall compute the two families of vectors $(p_i)_{1\le i\le u}$ and $(q_j)_{1\le j\le v}$. We then set ourselves a criterion to minimize:

$$(p_i)_{1\le i\le u}, (q_j)_{1\le j\le v} \longrightarrow \sum_{(i,j)\in\Omega} \big(grade_{ij} - \langle p_i, q_j\rangle\big)^2 = \|P_\Omega(A^* - PQ)\|_2^2$$

where $A^* = \big(grade_{ij}\big)_{i,j}$, $P = (p_i^\top)_{1\le i\le u}$, and $Q = (q_j)_{1\le j\le v}$. In matrix form, the objective function is written

$$P \in \mathbb{R}^{u\times d}, Q \in \mathbb{R}^{d\times v} \to \|P_\Omega(A^* - PQ)\|_2^2 . \tag{5.1}$$

This is the objective function that we shall seek to minimize. Nevertheless, since the product PQ is invariant under the change $(P,Q) \to (\alpha P, Q/\alpha)$ for every $\alpha \neq 0$, one risks having normalization problems if one merely minimizes (5.1). We shall therefore force the norms of the vectors (p_i) and (q_j) to be small as well, by regularizing by their norms: the objective function that we shall minimize is then

$$P \in \mathbb{R}^{u\times d}, Q \in \mathbb{R}^{d\times v} \to f(P,Q) := (1/2)\,\|P_\Omega(A^* - PQ)\|_2^2 + \lambda\left(\|P\|_2^2 + \|Q\|_2^2\right) \tag{5.2}$$

where $\lambda > 0$ is a regularization parameter to be chosen.

However, minimizing this function f jointly in (P,Q) is difficult, because the function f is not convex. Nevertheless, one can observe that

1. for a fixed P^*, $Q \to f(P^*, Q)$ is convex and its gradient is $\partial_2 f(P^*, Q) = -(P^*)^\top P_\Omega(A^* - P^*Q) + 2\lambda Q$
2. for a fixed Q^*, $P \to f(P, Q^*)$ is convex and its gradient is $\partial_1 f(P, Q^*) = -P_\Omega(A^* - PQ^*)(Q^*)^\top + 2\lambda P$

One can therefore alternate gradient descent steps in P and then in Q. This approach is sometimes called ARLS, for Alternating Ridge Least Squares.

Data: $\{A^*_{ij}, (i,j) \in \Omega\}$
Init: $P_0 \in \mathbb{R}^{u \times d}$, $Q_0 \in \mathbb{R}^{d \times v}$ and two sequences of step sizes $(\eta_k)_k$ and $(\mu_k)_k$
Output: $P \in \mathbb{R}^{u \times d}$ and $Q \in \mathbb{R}^{d \times v}$ such that $A^* \approx PQ$
while *stopping criteria* **do**
  $P_{k+1} = P_k - \eta_k \partial_1 f(P_k, Q_k)$
  $Q_{k+1} = Q_k - \mu_k \partial_2 f(P_{k+1}, Q_k)$
end

Figure 6: Alternating gradient descent algorithm for representing the matrix A^* as a product PQ.

At the output of algorithm 8, one obtains two matrices P and Q. One can use these matrices to build a recommender system: it then suffices to compute the product PQ and to identify the largest entries on every row. One can also cluster the users and the items in the latent space, simply by clustering the rows of P to find clusters of users and the columns of Q to find clusters of items. One may picture this idea with Figure 7.

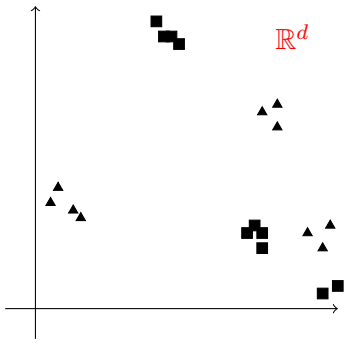


Figure 7: In addition to the design of a recommender system, the representation of users and items in the latent space can also be useful for clustering users and items according to their ratings. One can thus find groups of users with the same tastes relative to their ratings of these items (and similarly for the items).

Non-Negative Matrix Factorization (NMF). In the standard matrix factorization approach described above, the coordinates of the latent vectors p_i and q_j may be negative, and hence so may the inner product $\langle p_i, q_j \rangle$. In our approach, the inner product corresponds to a similarity between the user and the item. One may then take a negative inner product to mean that the user will probably not appreciate that item. This amounts to bringing this inner product back to the lowest rating, that is, to 1 in the case of Netflix. Another strategy is to impose positivity on the coordinates of the vectors p_i and q_j. One then adds a positivity constraint on the entries

of the matrices P and Q in our optimization problem. One obtains the following minimization procedure

$$\min_{(P,Q)\in\mathbb{R}^{u\times d}\times\mathbb{R}^{d\times v}} (\|P_\Omega(A^* - PQ)\|_2 : P \geq 0, Q \geq 0)$$

where we write "$P \geq 0$" when $P_{ij} \geq 0$, $\forall(i,j)$.

As in ARLS, the objective function is not convex in (P,Q), but it is convex in P for Q fixed, and also in Q for P fixed. We shall therefore alternate descents in P and then in Q. Except that, unlike the previous algorithm, one must respect the constraints "$P \geq 0$" and "$Q \geq 0$". To do so, one has the choice between **projected gradient** and **conditional gradient (also called the Frank–Wolfe algorithm)**. Since the projection onto the space of matrices with nonnegative entries is easy—it is $\text{proj} : P \to (\max(P_{ij}, 0))_{i,j}$—we shall use projected gradient.

Data: $\{A^*_{ij}, (i,j) \in \Omega\}$
Init: $P_0 \in \mathbb{R}^{u\times d}$, $Q_0 \in \mathbb{R}^{d\times v}$ and two sequences of step sizes $(\eta_k)_k$ and $(\mu_k)_k$
Output: $P \in \mathbb{R}^{u\times d}$ and $Q \in \mathbb{R}^{d\times v}$ such that $A^* \approx PQ$
while *stopping criteria* **do**
    $R_{k+1} = P_k - \eta_k \partial_1 f(P_k, Q_k)$
    $P_{k+1} = \text{proj}(R_{k+1})$
    $S_{k+1} = Q_k - \mu_k \partial_2 f(P_{k+1}, Q_k)$
    $Q_{k+1} = \text{proj}(S_{k+1})$
end

Figure 8: Alternating projected gradient descent algorithm for representing the matrix A^* as a product PQ in which P and Q have all entries nonnegative.

Other algorithms for NMF can be found in the paper of Lee and Seung, NIPS 2001.

Adding metadata to the matrix factorization algorithm. In many problems, in addition to users' ratings of items, one may have additional data called metadata (because they come in addition to the basic rating data). One may, for example, encounter data on the users (age, sex, socio-professional category, and so on), on the items (price, categories, and so on), or relations between users (membership in communities, likes, and so on), and likewise relations between items (same categories, release date of a movie, presence of actors, and so on). One imagines that adding these data when building a recommender system can only improve the quality of its recommendations. The aim of this section is to give ideas for modeling these data and to see how to include them in the optimization problems so that the construction of our recommender system takes them into account.

The idea behind matrix factorization for the design of recommender systems, mentioned above, is to explain the observed ratings $(A^*_{ij} : (i,j) \in \Omega)$ by constructing vectors (p_i) representing the users and vectors (q_j) representing the items in the same latent space $\mathbb{R}^d$, such that for all the observed ratings, that is, for every $(i,j) \in \Omega$, one has $A^*_{ij} \approx \langle p_i, q_j \rangle$. Metadata can also help explain the ratings $(A^*_{ij} : (i,j) \in \Omega)$. We shall therefore add the metadata to the model $A^*_{ij} \approx \langle p_i, q_j \rangle$. We consider metadata in the following form:

a) x_i is the feature vector of user i, for every $i \in [u]$ (autoencoding, embedding, one-hot encoding, and so on);

b) z_j is the feature vector of item j, for every $j \in [v]$ (autoencoding, embedding, one-hot encoding, and so on);

c) $U \in \mathbb{R}^{u \times u}$ is a users/users matrix representing the affinities/influences between users (for example, $U_{ij} = 1$ if i likes j and $U_{ij} = -1$ when i does not like j, or the number of "positive" messages exchanged between i and j, or the number of messages of j retweeted by i, and so on). The users/users matrix is here to indicate the influences between users; one wishes to encode in this matrix information of the type "if user r likes this movie, then this is also likely to be the case for user i". Thus, if j is a strong "influencer", then U_{ij} will be a large positive number for a majority of the other users i over whom j has influence,

d) $I \in \mathbb{R}^{v \times v}$ is an items/items matrix representing the affinities between items (for example, based on the cast, the producer, the year of release, the genre of two movies, and so on).

The idea is now to use these data to explain the observed ratings as well as possible. The first and most natural approach is to use linear models. We shall then explain the $(A^*_{ij} : (i,j) \in \Omega)$ in the following way: for every $(i,j) \in \Omega$,

$$A^*_{ij} \approx \langle p_i, q_j \rangle + (a_i + \langle b_i, x_i \rangle) + (c_j + \langle d_j, z_j \rangle) + \alpha \sum_{r=1}^{u} U_{ir} \langle p_r, q_j \rangle + \beta \sum_{s=1}^{v} I_{js} \langle p_i, q_s \rangle. \tag{5.3}$$

The model (5.3) takes into account the influences between users in the term "$\sum_{r=1}^{u} U_{ir} \langle p_r, q_j \rangle$" (if r influences i, then U_{ir} is a large positive number, and if r likes item j, then $\langle p_r, q_j \rangle$ will also be a large positive number; ultimately, $U_{ir} \langle p_r, q_j \rangle$ will be a large positive number, and hence the rating that i will give to j will be all the larger), the influences between items in the term "$\sum_{s=1}^{v} I_{js} \langle p_i, q_s \rangle$", the features specific to the users in the term "$a_i + \langle b_i, x_i \rangle$", and the features specific to the items in the term "$c_j + \langle d_j, z_j \rangle$".

The parameters of the model (5.3) that must be learned from our data are the latent vectors $(p_i)_i$ and $(q_j)_j$, and the parameters of the linear terms explaining the final rating from the features of the users and of the items, that is, a_i, b_i for $i \in [u]$ and c_j, d_j for $j \in [v]$. The learning algorithm that we shall run is also based on alternating minimizations between the vectors (p_i) and (q_j), together with an update of the coefficients a_i, b_i, c_j, d_j at each iteration. One may then either regularize by the ℓ_2^d norm of the p_i and q_j (as in ARLS), or impose a positivity constraint on the coordinates of the p_i and q_j, as in NMF. We give below the regularized version of the algorithm. We assume that $U_{ii} = 0$ and $I_{jj} = 0$, so that the information $\langle p_i, q_j \rangle$ is not redundant in the model (5.3). We denote by $F\big((p_i, a_i, b_i)_i, (q_j, c_j, d_j)_j\big)$ the quantity

$$\sum_{(i,j) \in \Omega} \left[A^*_{ij} - \left(\langle p_i, q_j \rangle + (a_i + \langle b_i, x_i \rangle) + (c_j + \langle d_j, z_j \rangle) + \alpha \sum_{r=1}^{u} U_{ir} \langle p_r, q_j \rangle + \beta \sum_{s=1}^{v} I_{js} \langle p_i, q_s \rangle \right) \right]^2 + \lambda \left(\sum_i \|p_i\|_2^2 + \sum_j \|q_j\|_2^2 \right)$$

which we wish to minimize. For $(p_i)_i$ and $(q_j)_j$ fixed, one can minimize F explicitly in (a_i, b_i, c_j, d_j), since one falls back on a least-squares problem.

6 Benchmark approaches: collaborative and content-based filtering

The approaches seen above are based on the low-rank assumption for the users/items matrix. It is this low-dimensional assumption on the signal A^* to be reconstructed that one seeks to encode, either directly in the solution space (by seeking the matrix of smallest rank among all matrices A such that $A_{ij} = A^*_{ij}$ for $(i,j) \in \Omega$), or by factorizing A^* as a low-rank product PQ. This approach, which may seem natural in view of the similarities between users and those between items, is nevertheless not the most natural one, and there are in fact algorithms that arise more naturally (and that were introduced at the very beginning of recommender systems, before those introduced above). These original algorithms, which are also based on the similarities between users and between items, are what are now called **benchmark algorithms**, because it is precisely to these algorithms that we must compare the performance of any new algorithm for building recommender systems. These algorithms are the simplest, and they require very little mathematical knowledge to understand and to implement. They are therefore also the fastest to run, which makes them all the more "benchmark". We present these algorithms in this section.

There are two types of benchmark algorithms for the design of recommender systems:

1. algorithms of **collaborative filtering** type, which use a similarity measure between users
2. algorithms of **content-based filtering** type, which use a similarity measure between items.

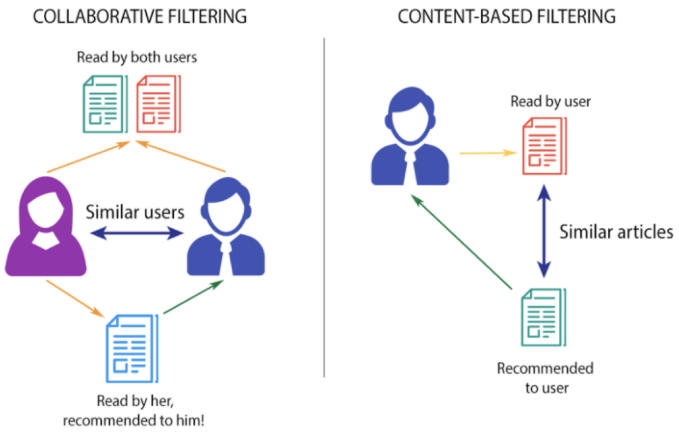


Figure 9: Two types of benchmark approaches for the design of recommender systems. Collaborative filtering is based on the similarities between users, and the content-based filtering approach is based on the similarities between items.

The whole point of collaborative and content-based filtering is to construct a similarity measure, either between users (for the collaborative filtering approach) or between items (for the content-based filtering approach):

a) for two users $i, i' \in [u]$, we denote by $\mathrm{sim}(i,i')$ a similarity measure between user i and user i'

b) for two items $j, j' \in [v]$, we denote by $\mathrm{sim}(j,j')$ a similarity measure between item j and item j'.

A recommender system of "collaborative filtering" type fills in the users/items matrix with

$$\hat{A}_{ij} = \frac{\sum_{i' \in [u]} \mathrm{sim}(i,i') \hat{A}_{i'j}}{\sum_{i' \in [u]} \mathrm{sim}(i,i')}$$

and a "content-based filtering" recommender system fills it in with

$$\hat{A}_{ij} = \frac{\sum_{j' \in [v]} \mathrm{sim}(j,j') \hat{A}_{ij'}}{\sum_{j' \in [v]} \mathrm{sim}(j,j')}.$$

The whole point is therefore indeed to define a similarity measure between users or between items. Along the way, one notes that in both formulas above, the definition of $\hat{A}_{ij}$ depends on the other entries $\hat{A}_{i'j}$ or $\hat{A}_{ij'}$, and one may therefore imagine some iterative algorithm. However, in the majority of collaborative or content-based filters, one performs only a single step, and in that case one sums only over the observed data A^*_{ij} for $(i,j) \in \Omega$.

The construction of similarity measures also lends itself very well to the addition of user, item, user/user, or item/item metadata. We shall not go into these details, and we give here only examples of similarity measures based on the observed ratings in the users/items matrix. For two users $i, i' \in [u]$, one uses the cosine similarity measure between the rating vectors of the two users, restricted to the items that both have rated:

$$\mathrm{sim}(i,i') = \frac{\sum_{j \in N(i,i')} A^*_{ij} A^*_{i'j}}{\sqrt{\sum_{j \in N(i,i')} (A^*_{ij})^2} \sqrt{\sum_{j \in N(i,i')} (A^*_{i'j})^2}}$$

where $N(i,i') = \{j \in [v] : (i,j), (i',j) \in \Omega\}$ is the set of items rated by both users. The numerator is the inner product $\left\langle (P_\Omega A^*)_{i\bullet}, (P_\Omega A^*)_{i'\bullet} \right\rangle$ of the masked rows of $P_\Omega A^*$, but the norms in the denominator are computed only on $N(i,i')$. For a similarity measure between items, one can define a cosine similarity measure in the same way.

7 Recommender systems by deep learning

Another approach that has received a great deal of attention in recent years is the construction of recommender systems by neural network models, also called *deep learning*. We present an example of a neural network architecture for the design of a recommender system in this section. There are many others. The one we present here is probably one of the simplest (the simplest being the one that builds a linear matrix factorization model). One may view this network as an elaboration of the matrix factorization approach seen above, because it uses embeddings of the users and of the items, except that, instead of embedding users and items in the same latent space and taking their inner product to predict the rating, one may embed users and items in two different latent spaces, and the rating will be predicted by a function more complicated than a simple inner product.

The embedding of users and items transforms a user represented by a one-hot encoding vector into a vector of a certain size, as represented in Figure 10. This is the first step, or layer, of the neural network that we shall construct.

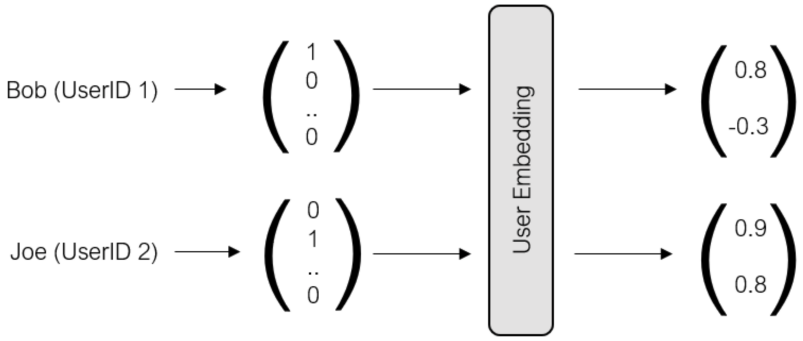


Figure 10: Vector representation of a user: "*users embedding*". It is a linear map that takes as input a one-hot encoding vector (a unit vector) of $\mathbb{R}^u$ and maps it to $\mathbb{R}^d$, where d is the dimension of the latent space to be chosen. There are $u \times d$ weights to be learned in this layer.

Embedding is an important component of every learning problem. It can be done by hand, which is what one customarily calls "featuring", or it can be done by means of a kernel, or it can be learned from the data. It is this last approach that we took above in the case of matrix factorization: the vectors p_i and q_i are embeddings of the users and of the items, which one learns by solving an optimization problem built from the data. It is also this approach that we shall use in the deep learning setting; the first layer of our neural network will be a layer of two embeddings, one for the users and one for the items.

The neural network that we shall construct below therefore begins with an embedding layer for the user and for the item; it is then followed by *fully connected* layers, also called *dense layers* or *deep fully connected* layers. The fully connected layer is the basic structure of neural networks, because it contains no structure (one says that it is "structured agnostic"). One is represented in Figure 11.

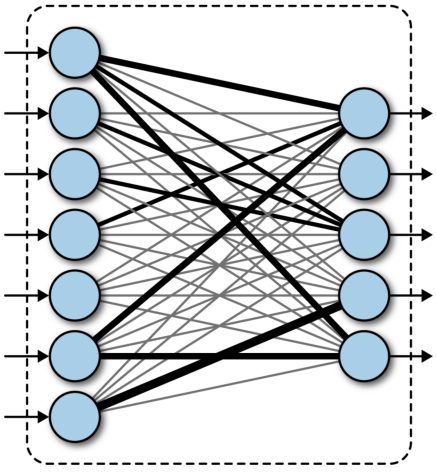


Figure 11: A fully connected layer of neurons with 9 units as input and 5 as output, for a total of 45 weights that will have to be learned.

Fully connected layers are used when one has no a priori knowledge of the structure of the problem one wishes to solve. This is the type of network one uses when one does not know what else to do. They are nevertheless expensive in training time, because they have the maximum

number of weights to fit, and they are in general less efficient than layers dedicated to a structured problem. Mathematically, a fully connected layer with m neurons as input and p neurons as output is a function parametrized by the weight matrix $W = (w_{ij} : 1 \leq i \leq p, 1 \leq j \leq m)$, which, given an input vector $x \in \mathbb{R}^m$, returns a vector $y \in \mathbb{R}^p$ such that for every $i \in [p]$

$$y_i = \sigma\left(w_{i1}x_1 + \cdots + w_{im}x_m\right)$$

where $\sigma : \mathbb{R} \to \mathbb{R}$ is an activation function, such as the sigmoid function $\sigma : t \to (1 + \exp(-t))^{-1}$ or the ReLU (Rectified Linear Unit) function $\sigma : t \to \max(0, t)$. Fully connected layers can be stacked on one another if one wishes to learn complicated dependence structures between the inputs and the outputs. Figure 12 represents a network of 4 fully connected networks.

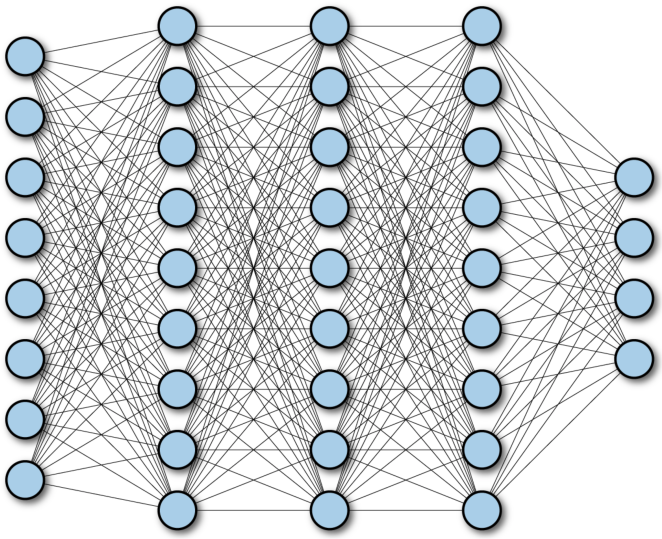


Figure 12: A network architecture of 4 fully connected networks, with respectively 8 neurons as input, 4 as output, and 9, 9, and 9 neurons for the intermediate layers. This gives a total of $8 \times 9 + 9 \times 9 + 9 \times 9 + 9 \times 4 = 270$ weights to be learned.

The neural network used in NCF (neural collaborative filtering) is represented in Figure 14. It takes as input pairs (user, item) and as output it returns a probability vector over (here) the two possible choices "interaction" (i.e., the output equals 1) or "no interaction" (that is, the output equals 0). This output vector is then passed through a sigmoid, which returns the most probable class; here class 1 is returned because $0.8 > 0.2$.

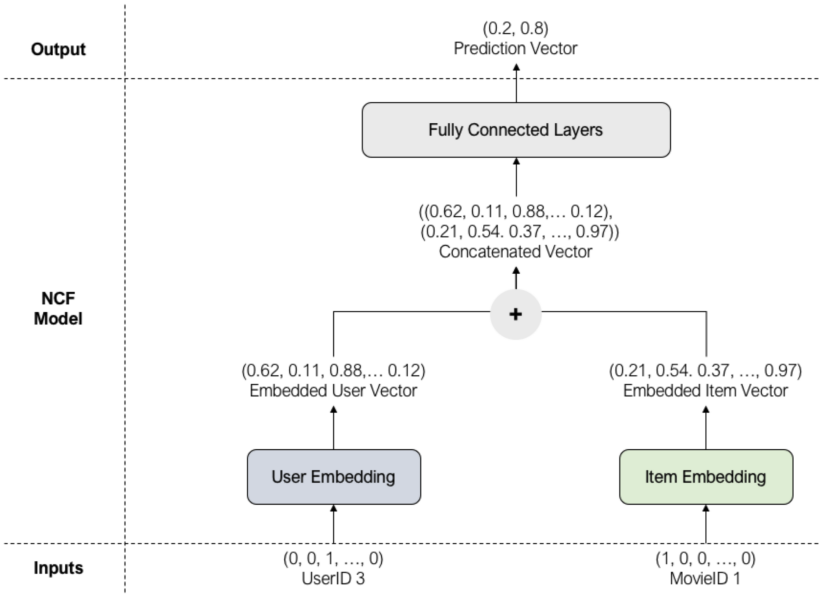


Figure 13: Architecture of *Neural Collaborative Filtering*: an embedding layer followed by a fully connected network giving the vector of probabilities of belonging to the two classes (interaction or not).

Now that we have decided on the structure of the neural network that we wish to use, we must learn the weights that compose it from the data. We must first choose a loss function. Here we take the **Binary Cross Entropy** (BCE), which, to a batch $(user_i, item_i, y_i)_{i=1}^N$, associates a loss

$$BCE(\hat{y}, y) = -\sum_{i=1}^{N} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

where $\hat{y} = (\hat{y}_i)_{i=1}^N$ is the vector of predictions at the output of the network for the successive inputs $(user_i, item_i)$ for $i = 1, \ldots, N$. The loss $BCE(\hat{y}, y)$ is a function of all the weights that compose the NCF neural network. We denote these weights by w. We therefore have

$$BCE(\hat{y}, y) = f(w, (user_i, item_i)_{i=1}^N) = f(w).$$

We shall learn the weights w by minimizing the function $w \to f(w)$, by taking several steps of an optimization method inspired by gradient descent. The solver currently most widely used to learn the weights of neural networks is called ADAM (Adaptive moment estimation), which is a first-order method. ADAM performs the following update on the weights. Set $m_0 = 0$ and $v_0 = 0$, and choose $\beta_1, \beta_2, \eta, \varepsilon \in (0, 1)$. One computes estimates of the first two moments:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) \nabla f(w^{(k)}) \quad \text{and} \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2)(\nabla f(w^{(k)}))^2 \,,$$

then one corrects the bias

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k} \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k} \,,$$

and one performs the following update of the weights with

$$w^{(k+1)} = w^{(k)} - \frac{\eta}{\sqrt{\hat{v}_k} + \varepsilon} \hat{m}_k. \tag{7.1}$$

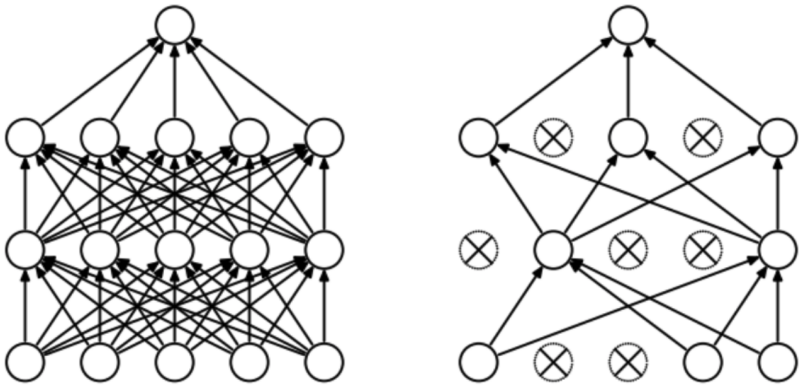
The key point of every first-order method is to be able to compute efficiently the gradient of a function that may be very complicated (for example, f depends on at least 270 variables if the network of Figure 12 is used in NCF). This is where the **backpropagation** algorithm comes into play. It is an algorithm that comes from the theory of **automatic differentiation**, and that makes it possible, chiefly by means of the chain rule, to compute gradients of a complicated function at a given point. It is this backpropagation algorithm that is used to compute the gradients for any solver, in particular for ADAM.

Finally, there are many parameters to choose when one trains a neural network, such as the number of layers, the number of neurons, the dimensions of the embedding spaces, the number of **epochs** (this is the number of times one passes entirely over the database, split into batches, in order to learn the weights of the network), the batch size, and the learning rate (this is the parameter η in (7.1)). Logically, one should learn them with cross-validation criteria; however, the training time of a single network is too costly to implement this strategy. One then uses "best practice" approaches, which come to us from the intuitions or the practices of experts, who tell us, for example, to take a batch size of 512, for 5 epochs, and to take a dense layer with 16 neurons as input, 64 intermediate neurons, and 32 as output.

Once the network $f(\hat{w}, (\cdot, \cdot))$ has been learned, one can fill in the users/items matrix by querying all the unobserved entries through $f(\hat{w}, (user_i, item_j))$ for $(i, j) \notin \Omega$.

Negative sample. There is nevertheless an important point that one systematically encounters when building recommender systems via a neural network. In fact, in the example considered above, the database is of the form $(user_i, item_i)_{i=1}^n$; here there is no rating, since one looks only at the users and items that have interacted. In particular, one has no example of a (user, item) pair that did not interact by choice, because one cannot distinguish an absence of interaction because the user did not think of interacting with that item (whereas, had they thought of it, they would have interacted with it) from an absence of interaction because the user does not wish to interact with that item. What are called **negative samples** are therefore missing, that is, examples of interactions with items deliberately avoided by the user. One must then find a way to generate negative examples. Otherwise, one will give the network only positive examples to learn, and it will therefore learn to answer systematically 1 for every pair $(user, item)$. The simplest way, however widely debatable, is simply to pick at random items among those with which the user has not interacted, and to add them as negative samples. In doing so, one falls back on the problem seen earlier, namely that one has not interacted by choice or by lack of knowledge of the existence of that choice. Nevertheless, this is the simplest approach, and it is the one we shall implement (one can also look at the least popular items by batch, or at other, more collaborative strategies for creating negative samples). Here, for example, one may choose 4 times as many negative samples as positive samples per user, in order to compensate for the "poor" quality of our negative samples.

Dropout. There is still much to be done concerning the understanding of the overfitting phenomenon in neural networks (how can an algorithm that makes zero error on the training sample still have a very good ability to generalize?). In the (still persistent) spirit of avoiding overfitting of neural networks, the idea of dropout has made its way. The principle is to consider, at random, a certain proportion of the units, or neurons, during training (at each gradient descent step, one chooses new ones at random) and not to optimize the weights associated with these units. This introduces an additional parameter: the proportion p of units to be chosen at random at each iteration, which must also be chosen in advance. The principle of dropout is represented in the Figure



(a) Standard Neural Net (b) After applying dropout.

Figure 14: Dropout: an idea for avoiding overfitting of neural networks (see Srivastava, Nitish, et al., JMLR 2014).

8 Performance criteria

Given an observed subset of observed ratings $(A^*_{ij} : (i,j) \in \Omega)$, one sets aside a certain percentage of this database, which will be used as the "test set"; the remaining ratings are used as the "training set". One does not choose completely at random the ratings set aside for the test set throughout the matrix: one must ensure that each row (that is, each user) has at least about fifty ratings present in the test set.

Assume that a recommender system has been constructed. For each user, one identifies the set of items that they did in fact appreciate in the test set, and one counts among them the number of recommendations that one makes to them. One divides this number by the total number of items that were genuinely appreciated. This gives the **recall**. For the **precision**, one does the same thing, except that one divides by the number of recommended items. These two quantities are given for each user. One can aggregate them in multiple ways (by taking their mean and quantiles, for example), or one can compute the F1 score, given for each user by

$$F1 = \frac{2 \times recall \times precision}{recall + precision}.$$

One can then represent the distribution of these F1 scores.

One can also consider metrics of the type **MAE** (mean absolute error), which is the ℓ_1 norm between the estimated ratings and the ratings of the test set, or the ℓ_2 norm, which gives the **RMSE** (root mean square error). In fact, any distance between the estimated ratings and the observed ratings of the test set can be used as a performance criterion. It is always good to check these performance criteria both on the test set and on the learning set, in order to see whether there has been too much overfitting (which occurs when the error on the training set is much smaller than the error on the test set). This "measure" of overfitting has nevertheless been called into question fairly recently, with deep learning methods that manage to make no error on the training sample (which was considered a typical case of overfitting before) while still generalizing well, that is, while making a reasonable error on the test sample.